

Esercizi d'esame per categoria — Parte B (2020–2026)

Raccolta di **tutti i 99 esercizi** dei 33 esami di *Parte B* presenti in [Esami/Seconda parte](#) (dal 14-01-20 al 02-02-26), raggruppati per **tecnica risolutiva** anziché per data. Ogni esercizio riporta la traccia riassunta, l'idea della soluzione ottima con la formula/ricorrenza chiave, la complessità e il riferimento all'esame di provenienza (es. [\[24-02-01 · B3\]](#)).

Indice

B. Programmazione dinamica e memoization — 41 esercizi

- [B.1 Conteggio di composizioni e tassellazioni \(Fibonacci / k-nacci / Motzkin\)](#)
- [B.2 Conteggio di alberi binari \(numeri di Catalan e varianti\)](#)
- [B.3 Conteggio e ottimo su griglia / sottomatrici](#)
- [B.4 Conteggio di sottosequenze e sottoinsiemi](#)
- [B.5 DP su stringhe \(famiglia LCS\)](#)
- [B.6 DP tipo zaino / subset-sum](#)
- [B.7 DP di selezione su sequenza con vincoli](#)
- [B.8 DP di partizionamento / parentesizzazione](#)

C. Grafi — 1 esercizio

- [C.1 Cammini minimi](#)

D. Flusso massimo e matching — 13 esercizi

- [D.1 Assegnamento bipartito con capacità \(lo schema base\)](#)
- [D.2 Reti con gadget: quote per categoria e diversità](#)
- [D.3 Node splitting e cammini disgiunti](#)
- [D.4 Test di robustezza / ricalcolo](#)

E. Greedy — 7 esercizi

- [E.1 Ordinamento e argomento di scambio \(greedy corretti\)](#)
- [E.2 Greedy che NON funziona \(controesempi da costruire\)](#)

F. Backtracking ed enumerazione — 33 esercizi

- [F.1 Stringhe binarie / n-arie con vincoli sui caratteri consecutivi](#)
- [F.2 Sottosequenze e sottoinsiemi con vincolo d'ordine / distanza](#)
- [F.3 Composizioni e somme di interi](#)
- [F.4 Permutazioni con vincoli](#)
- [F.5 Cammini e cicli su griglie e grafi](#)
- [F.6 Cammini radice-foglia su alberi](#)

G. Array, scansioni, ricerca — 4 esercizi

Appendice A: indice per esame

Appendice B: esercizi di Parte A ricorrenti

B. Programmazione dinamica e memoization

La categoria più frequente in assoluto (41 esercizi su 99, **41%**): quasi ogni esame ha almeno un esercizio di DP, spesso in posizione **B3** (il più pesante in punti). Da scrivere sempre: (1) definizione **a parole** del significato della tabella, (2) **formula ricorsiva** coi casi base, (3) codice (iterativo bottom-up o con memoization), (4) complessità, (5) eventualmente la ricostruzione. In questo periodo il tema più martellato è il **conteggio del numero di soluzioni** (sotto-categorie B.1–B.4): la stessa idea "conta in quanti modi..." ritorna sotto travestimenti diversi (tassellazioni, alberi, cammini, sottosequenze) e produce famiglie note (Fibonacci, Tetranacci, Catalan, Motzkin).

Scheletro tipico vs diverso. Qui lo "scheletro" è la **ricorrenza + riempimento della tabella** (bottom-up o memoization): la formula già riportata è la soluzione e il codice è meccanico, quindi **hanno scheletro tipico tutti gli esercizi tranne due**, che richiedono strutture ausiliarie o cicli non ovvi e per questo riportano lo pseudocodice per esteso — **Largest square** ([20-02-04 · B3], pre-calcolo delle matrici R/C dei run) e **Borsa** ([26-01-12 · B3], ciclo interno sul giorno d'acquisto). (Caso limite: *minSteps* [25-02-03 · B3] ha transizioni su un intervallo di celle, ma la ricorrenza le cattura per intero → tipico, memoization diretta.) Inoltre, per lo **schema base** di ogni sotto-categoria (B.1–B.8) è riportato qui sotto il codice in **entrambe** le versioni — bottom-up e memoization — a scopo di studio.

B.1 Conteggio di composizioni e tassellazioni

Schema base ("modi di scendere una scala"): $T(n) = \sum_{k=1}^s T(n-k)$ con $T(0) = 1$. Al variare del numero s di passi ammessi si ottengono Fibonacci ($s = 2$), Tetranacci ($s = 4$), k -nacci in generale. Riconoscere questa forma è il primo riflesso da avere davanti a "in quanti modi...".

Griglia lineare $1 \times n$ con pezzi $1 \times 1, \dots, 1 \times s$ [23-01-17 · B1]

Traccia. Contare in quanti modi si riempie una griglia $1 \times n$ usando pezzi di lunghezza da 1 a s .

Idea. Ogni tassellazione è individuata **univocamente** dalla lunghezza $k \in \{1, \dots, s\}$ del **primo** pezzo (quello appoggiato a sinistra): rimosso quel pezzo, resta da riempire una griglia $1 \times (n-k)$, sottoproblema identico. Sommando su tutte le lunghezze iniziali possibili si contano tutte le tassellazioni, ciascuna una volta sola. Il caso base $T(0) = 1$ è la griglia vuota (un unico modo: non mettere nulla).

$$T(n) = \begin{cases} 1 & n = 0 \\ \sum_{k=1}^{\min(n,s)} T(n-k) & n > 0 \end{cases}$$

Complessità. $O(n \cdot s)$. Per $s = 4$ è la successione di **Tetranacci**.

Bottom-up.

```
tassellazioni(n, s):                                # modi di riempire 1xn con pezzi 1..s
    T[0] = 1                                          # griglia vuota: un solo modo
    for i = 1 to n:
        T[i] = 0
        for k = 1 to min(i, s):
            T[i] = T[i] + T[i-k]
    return T[n]
```

Memoization.

```
tassellazioni(n, s):
    for i = 0 to n: T[i] = NIL                      # ⊥ = non ancora calcolato
    return solve(n, s, T)
```

```

solve(i, s, T):
    if i == 0: return 1          # caso base
    if T[i] != NIL: return T[i]  # già calcolato
    T[i] = 0
    for k = 1 to min(i, s):
        T[i] = T[i] + solve(i-k, s, T)
    return T[i]

```

Dollar word: stringhe di valore T centesimi [25-06-13 · B2]

Traccia. Alle lettere **a...z** si associano i valori 1..26. Contare quante stringhe (di lunghezza arbitraria, ordine rilevante) valgono esattamente T centesimi.

Idea. Identico allo schema "modi di scendere una scala": si fissa il **valore della prima lettera** $i \in \{1, \dots, 26\}$; il resto della stringa deve valere $T - i$, sottoproblema identico. Poiché l'ordine conta, prime lettere diverse producono stringhe diverse $\rightarrow$ si sommano i $DP[t - i]$. Caso base $DP[0] = 1$ (la stringa vuota vale 0, in un solo modo).

$$DP[t] = \begin{cases} 1 & t = 0 \\ \sum_{i=1}^{\min(26, t)} DP[t - i] & t > 0 \end{cases}$$

È il caso $s = 26$ del precedente (numeri di **Hexacosanacci**). **Complessità.** $\Theta(T)$ (il ciclo interno è limitato dalla costante 26). ⚠ Non farsi ingannare: i primi valori sono potenze di 2, ma dal 27° termine no.

Going nowhere: sequenze di n mosse L/R/S che tornano a 0 [25-08-25 · B3]

Traccia. Una pedina parte da 0 su una retta infinita; ogni mossa è -1 (L), $+1$ (R) o 0 (S). Contare le sequenze di n mosse che riportano a 0.

Idea. $DP[s][p]$ = numero di modi di tornare in 0 con s mosse **rimaste**, partendo dalla posizione p . La prima mossa porta in $p - 1$, p o $p + 1$: si sommano i tre sottoproblemi con una mossa in meno. Potatura chiave: se $|p| > s$ non si può più annullare la distanza ($DP = 0$), quindi bastano le posizioni con $|p| \leq \lfloor n/2 \rfloor$.

$$DP[s][p] = \begin{cases} 0 & |p| > s \\ 1 & s = 0 \wedge p = 0 \\ DP[s-1][p-1] + DP[s-1][p] + DP[s-1][p+1] & \text{altrimenti} \end{cases}$$

dove p è la posizione, s i passi rimasti. Basta memoizzare su $|p| \leq \lfloor n/2 \rfloor$. **Complessità.** $\Theta(n^2)$. È la successione dei **Grand Motzkin path**.

Double sequence: sequenze crescenti "a raddoppio" [22-06-21 · B3]

Traccia. Contare le sequenze di n interi positivi $\leq m$ in cui ogni elemento è $\geq$ del doppio del precedente.

Idea. $DP[i][j]$ = numero di sequenze (crescenti "a raddoppio") lunghe i con **ultimo elemento** $\leq j$. Due casi disgiunti: le sequenze il cui ultimo elemento è $\leq j - 1$ ($DP[i][j - 1]$) e quelle il cui ultimo elemento è **esattamente** j — in tal caso il penultimo dev'essere $\leq \lfloor j/2 \rfloor$ (perché $j \geq 2 \cdot \text{penultimo}$), cioè $DP[i - 1][\lfloor j/2 \rfloor]$. Casi base: $j = 0$ non ammette valori positivi (0); con lunghezza $i = 1$ ci sono j scelte ($1, \dots, j$).

$$DP[i][j] = \begin{cases} 0 & j = 0 \\ j & i = 1 \\ DP[i - 1][\lfloor j/2 \rfloor] + DP[i][j - 1] & \text{altrimenti} \end{cases}$$

$DP[i][j]$ = sequenze lunghe i con valori $\leq j$. **Complessità.** $O(nm)$.

Math ways: modi di ottenere n da k con $+1, \times 2, \cdot^2$ [22-02-07 · B3]

Traccia. Contare i modi distinti di trasformare k in n con le operazioni "aggiungi 1", "raddoppia", "eleva al quadrato".

Idea. Si ragiona **in avanti**: $DP[i]$ = numero di modi di arrivare a n partendo dal valore corrente i . Da i le tre mosse portano a $i + 1, 2i, i^2$: si sommano i modi da ciascuno di questi tre stati. Poiché le operazioni **non decrescono mai** il valore, appena si supera n ($i > n$) lo stato è irrecuperabile (0); arrivati esattamente a n si è trovato un modo (1). Non ci sono doppioni perché ogni sequenza di mosse corrisponde a un cammino distinto in questo grafo aciclico.

$$DP[i] = \begin{cases} 0 & i > n \\ 1 & i = n \\ DP[i+1] + DP[2i] + DP[i^2] & \text{altrimenti} \end{cases}$$

Risposta. $DP[k]$. **Complessità.** $\Theta(n)$. (La versione "stampa tutti i modi" è in F.3, [22-06-21 · B1].)

Somme palindrome pari (conteggio) [21-08-31 · B3]

Traccia. Contare le somme palindrome di lunghezza **pari** che danno n (es. $6 = 1+1+1+1+1+1 = 1+2+2+1 = 2+1+1+2 = 3+3$: 4 modi).

$$DP[i] = \begin{cases} 1 & i = 0 \\ \sum_{j=1}^{i/2} DP[i-2j] & i > 0 \text{ (pari)} \end{cases}$$

Ogni coppia palindroma "esterna" vale j e consuma $2j$. **Complessità.** $O(n^2)$. (La versione "stampa tutte le somme palindrome" è in F.3, [20-06-12 · B3].)

B.2 Conteggio di alberi binari

Tutti questi si riducono a una **ricorrenza sui sottoalberi**: si consuma un nodo per la radice e si distribuiscono gli $n - 1$ restanti fra sottoalbero sinistro e destro, moltiplicando i conteggi. Da questo nascono i **numeri di Catalan** (alberi qualunque) e le loro varianti coi vincoli.

Numero di alberi con una data visita simmetrica = Catalan [21-01-18 · B3]

Traccia. Contare gli alberi strutturalmente diversi di n nodi la cui visita simmetrica dà $1, 2, \dots, n$. (La numerazione è forzata dalla struttura: si conta solo la forma.)

Idea. Si sceglie **quanti nodi mettere nel sottoalbero sinistro**: se sono k (con $0 \leq k \leq n - 1$), la radice ne consuma 1 e nel destro restano $n - 1 - k$. Le due forme sono indipendenti, quindi si **moltiplicano** i conteggi $DP[k] \cdot DP[n - 1 - k]$; sommando su k si ottengono tutte le forme senza doppioni. Caso base: 0 o 1 nodo $\rightarrow$ un unico albero.

$$DP[n] = \begin{cases} 1 & n \leq 1 \\ \sum_{k=0}^{n-1} DP[k] \cdot DP[n-1-k] & n > 1 \end{cases}$$

Complessità. $\Theta(n^2)$. È l' n -esimo **numero di Catalan** $C_n = \frac{1}{n+1} \binom{2n}{n}$.

Bottom-up.

```
catalan(n):
    DP[0] = 1; DP[1] = 1          # 0 o 1 nodo: un solo albero
    for i = 2 to n:
        DP[i] = 0
        for k = 0 to i-1:        # k nodi a sinistra, i-1-k a destra
            DP[i] = DP[i] + DP[k] * DP[i-1-k]
    return DP[n]
```

Memoization.

```
catalan(n):
    for i = 0 to n: DP[i] = NIL
    return solve(n, DP)

solve(i, DP):
    if i <= 1: return 1
    if DP[i] != NIL: return DP[i]
    DP[i] = 0
    for k = 0 to i-1:
        DP[i] = DP[i] + solve(k, DP) * solve(i-1-k, DP)
    return DP[i]
```

Quante parentesi: parentesizzazioni corrette di n coppie [25-01-14 · B3]

Traccia. Contare le stringhe ben bilanciate di n coppie di parentesi. Stessa ricorrenza di Catalan, letta come "prima coppia che racchiude i coppie, seguita da $n - i - 1$ ":

$$DP[k] = \sum_{i=0}^{k-1} DP[i] \cdot DP[k-i-1], \quad DP[0] = DP[1] = 1$$

Complessità. $O(n^2)$. (La versione "stampa tutte" è in F.1, [21-01-18 · B1].)

Alberi ben-bilanciati [22-01-17 · B3]

Traccia. Contare gli alberi di n nodi in cui in **ogni** nodo i due sottoalberi differiscono al più di 1 nel numero di nodi.

$$DP[i] = \begin{cases} 1 & i \leq 1 \\ DP[(i-1)/2]^2 & i \geq 2, i \text{ dispari} \\ 2 DP[(i-1)/2] \cdot DP[(i-1)/2] & i \geq 2, i \text{ pari} \end{cases}$$

Se i è dispari i due lati hanno per forza $(i-1)/2$ nodi ciascuno; se è pari, il nodo in più sta a sinistra o a destra (fattore 2). **Complessità.** $\Theta(n)$ — i sottoproblemi sono solo su indici dimezzati.

Alberibelli: alberi in cui ogni nodo ha 0 o 2 figli [24-02-01 · B2]

Traccia. Contare gli alberi binari di n nodi in cui **nessun nodo ha esattamente 1 figlio** (full binary tree).

Idea. Come Catalan, ma con $n > 1$ la radice deve avere **entrambi** i figli non vuoti: si distribuiscono gli $n - 1$ nodi restanti fra sinistra (j nodi) e destra ($n - 1 - j$ nodi) con j da 1 a $n - 2$ (nessun lato può essere vuoto), moltiplicando i conteggi. Ne segue $DP[n] = 0$ per n pari — servirebbe un lato con un numero pari di nodi, ma un full binary tree ha sempre un numero **dispari** di nodi.

$$DP[n] = \begin{cases} 1 & n = 1 \\ 0 & n = 2 \\ \sum_{j=1}^{n-2} DP[j] \cdot DP[n-1-j] & n > 2 \end{cases}$$

Correttamente $DP[n] = 0$ per n pari (uno dei due lati avrebbe un numero pari di nodi). **Complessità.** $O(n^2)$.

Made-up: alberi "ma come se li inventa" (vincolo di parità sui livelli) [23-07-11 · B3]

Traccia. Contare alberi in cui la radice a **livello pari** ha 0 o 2 figli, a **livello dispari** ha esattamente 1 figlio (a sinistra o a destra). Serve tenere la parità del livello nello stato:

Idea. La regola dipende dalla **parità del livello**, quindi lo stato è "quanti nodi" + "parità del livello della radice del sottoalbero". Un nodo a livello **pari** si comporta come un full binary (0 o 2 figli → si spezzano gli $n - 1$ nodi fra due sottoalberi, ora a livello **dispari**); un nodo a livello **dispari** ha esattamente 1 figlio, che può stare a sinistra o a destra (fattore 2) ed è a livello **pari**. Le due ricorrenze si alternano; basta ricordare la sola parità, non il livello esatto.

$$DP_{\text{pari}}[n] = \begin{cases} 1 & n = 1 \\ \sum_{i=1}^{n-2} DP_{\text{disp}}[i] \cdot DP_{\text{disp}}[n-1-i] & n > 1 \end{cases} \quad DP_{\text{disp}}[n] = \begin{cases} 0 & n = 1 \\ 2 DP_{\text{pari}}[n-1] & n > 1 \end{cases}$$

Si comprime il livello ai soli due valori pari/dispari (due vettori). **Risposta.** $DP_{\text{pari}}[n]$. **Complessità.** $O(n^2)$.

B.3 Conteggio e ottimo su griglia / sottomatrici

Numero di cammini su griglia $n \times n$ (solo destra/basso) [20-06-12 · B2]

Traccia. Contare i cammini da $(1, 1)$ a (n, n) muovendosi solo a destra o in basso.

Idea. $DP[i][j]$ = numero di cammini che arrivano in (i, j) . L'ultima mossa è stata da **sinistra** o dall'**alto**: i due insiemi di cammini sono disgiunti, quindi si sommano $DP[i][j-1] + DP[i-1][j]$. Sul bordo (prima riga/colonna) c'è un solo cammino (tutto dritto).

$$DP[i][j] = \begin{cases} 1 & i = 0 \vee j = 0 \\ DP[i-1][j] + DP[i][j-1] & \text{altrimenti} \end{cases}$$

Complessità. $\Theta(n^2)$. Il risultato è $\binom{2n}{n}$.

Bottom-up.

```
countPaths(n):
  for i = 1 to n: DP[i][1] = 1          # bordi: un solo cammino dritto
  for j = 1 to n: DP[1][j] = 1
  for i = 2 to n:
    for j = 2 to n:
      DP[i][j] = DP[i-1][j] + DP[i][j-1]
  return DP[n][n]
```

Memoization.

```
countPaths(n):
  for i = 1 to n, j = 1 to n: DP[i][j] = NIL
  return solve(n, n, DP)
```

```
solve(i, j, DP):
    if i == 1 or j == 1: return 1          # prima riga/colonna
    if DP[i][j] != NIL: return DP[i][j]
    DP[i][j] = solve(i-1, j, DP) + solve(i, j-1, DP)
    return DP[i][j]
```

Corsa a ostacoli: cammini su griglia con celle vietate [24-06-14 · B1]

Traccia. Come sopra ma da (n, n) a $(1, 1)$ (mosse su/sinistra) e alcune celle sono **false** (non attraversabili).

Idea. Stessa somma "da sinistra + dall'alto" del conteggio su griglia, con una sola aggiunta: una cella vietata azzerà i cammini che la attraversano ($DP = 0$ se $A[i][j] = \text{false}$), così nessun cammino può passarci. Il resto propaga i conteggi come al solito.

$$DP[i][j] = \begin{cases} 0 & A[i][j] = \text{false} \vee i = 0 \vee j = 0 \\ 1 & i = 1 \wedge j = 1 \wedge A[i][j] \\ DP[i-1][j] + DP[i][j-1] & \text{altrimenti} \end{cases}$$

Complessità. $O(n^2)$.

Quanti quadrati: contare le sottomatrici quadrate di soli 1 [24-01-09 · B3]

Traccia. Contare **tutte** le sottomatrici quadrate composte da soli 1.

Soluzione ($O(n^2)$). $DP[i][j]$ = lato del più grande quadrato di 1 con **angolo in basso a destra** in (i, j) :

$$DP[i][j] = \begin{cases} 0 & A[i][j] = 0 \\ 1 & A[i][j] = 1 \wedge (i = 1 \vee j = 1) \\ \min\{DP[i-1][j], DP[i-1][j-1], DP[i][j-1]\} + 1 & \text{altrimenti} \end{cases}$$

Poiché un quadrato di lato $DP[i][j]$ implica anche quadrati di lato $1, \dots, DP[i][j]$ che terminano in (i, j) , la risposta è la **somma dell'intera matrice DP**. **Complessità.** $\Theta(n^2)$. (Sono note anche soluzioni $O(n^5)$, $O(n^4)$, $O(n^3)$: la traccia le chiede esplicitamente.)

Largest square: più grande sottomatrice con **bordi** di 1 [20-02-04 · B3]

Traccia. Trovare il lato della più grande sottomatrice quadrata con **i soli bordi** di valore 1.

Idea. Il costo di un quadrato di bordi sta nel verificare i suoi 4 lati; farlo scorrendo le celle è troppo lento, quindi si pre-calcolano due matrici che rispondono in $O(1)$ a "quanti 1 consecutivi ci sono?": $R[i][j]$ = lunghezza del run **orizzontale** di 1 che termina in (i, j) , $C[i][j]$ = run **verticale**. Un quadrato di lato k con angolo in basso a destra in (i, j) esiste sse e solo se i suoi 4 lati sono pieni: lato basso $R[i][j] \geq k$, lato destro $C[i][j] \geq k$, lato alto $R[i-k+1][j] \geq k$, lato sinistro $C[i][j-k+1] \geq k$. Si prova ogni cella e ogni lato k ; l'ottimizzazione "non testare lati $\leq \text{maxSoFar}$ " evita lavoro inutile.

Soluzione ($O(n^3)$). Si pre-calcolano in $O(n^2)$ le matrici R (run orizzontale di 1 che finisce in (i, j)) e C (run verticale). Poi per ogni cella e ogni lato k si verificano i quattro lati in $O(1)$ con R, C . **Complessità.** $O(n^3)$.

```
largestSquare(A, n):
    for i = 1 to n:
        # pre-calcolo dei run
        R[i][1] = A[i][1]; C[1][i] = A[1][i]
    for i = 2 to n, j = 2 to n:
        R[i][j] = (A[i][j]==0) ? 0 : R[i][j-1] + 1
        C[i][j] = (A[i][j]==0) ? 0 : C[i-1][j] + 1
    maxSoFar = 0
```

```

for i = 1 to n, j = 1 to n:
    m = min(R[i][j], C[i][j])           # lato massimo possibile qui
    for k = maxSoFar+1 to m:           # ← salto i lati già battuti
        if R[i-k+1][j] >= k and C[i][j-k+1] >= k: # lato alto e sinistro pieni
            maxSoFar = max(maxSoFar, k)
return maxSoFar

```

B.4 Conteggio di sottosequenze e sottoinsiemi

countMenu: sottoinsiemi con somma k (conteggio) [20-09-04 · B1]

Traccia. Contare i sottoinsiemi di A (ogni piatto al più una volta) con somma esattamente k .

Idea. Classico "**prendo / non prendo**" l'elemento i . $DP[i][r]$ = numero di sottoinsiemi dei primi i elementi con somma r . Se non prendo $A[i]$: $DP[i-1][r]$. Se lo prendo (possibile solo se $A[i] \leq r$): resta da fare somma $r - A[i]$ coi primi $i-1$, cioè $DP[i-1][r - A[i]]$. I due casi sono disgiunti → si sommano. Casi base: somma 0 raggiungibile in un modo (insieme vuoto); somma > 0 senza elementi in 0 modi.

$$DP[i][r] = \begin{cases} 1 & r = 0 \\ 0 & r > 0 \wedge i = 0 \\ DP[i-1][r] + DP[i-1][r - A[i]] & A[i] \leq r \\ DP[i-1][r] & A[i] > r \end{cases}$$

Risposta. $DP[n][k]$. **Complessità.** $O(nk)$ (pseudopolinomiale). Piatti con stesso costo vengono contati separatamente.

Bottom-up.

```

countMenu(A, n, k):
    DP[0][0] = 1
    for r = 1 to k: DP[0][r] = 0
    for i = 1 to n:
        for r = 0 to k:
            DP[i][r] = DP[i-1][r]
            if A[i] <= r:
                DP[i][r] = DP[i][r] + DP[i-1][r-A[i]]
    return DP[n][k]

```

Memoization.

```

countMenu(A, n, k):
    for i = 0 to n, r = 0 to k: DP[i][r] = NIL
    return solve(A, n, k, DP)

solve(A, i, r, DP):
    if r == 0: return 1
    if i == 0: return 0
    if DP[i][r] != NIL: return DP[i][r]
    DP[i][r] = solve(A, i-1, r, DP)
    if A[i] <= r:
        DP[i][r] = DP[i][r] + solve(A, i-1, r-A[i], DP)
    return DP[i][r]

```

productK: sottosequenze con prodotto $\leq m$ [23-02-09 · B3]

Traccia. Contare le sottosequenze di A (interi positivi) il cui **prodotto** è $\leq m$ (la sottosequenza vuota ha prodotto 1).

Idea. Stesso "prendo / non prendo" del subset-sum, ma **moltiplicativo**. $DP[i][k]$ = numero di sottosequenze dei primi i elementi con prodotto $\leq k$. Se scarto $A[i]$: $DP[i-1][k]$. Se lo includo (solo se $A[i] \leq k$): il prodotto dei restanti deve stare $\leq \lfloor k/A[i] \rfloor$, cioè $DP[i-1][\lfloor k/A[i] \rfloor]$ (divisione intera perché i valori sono interi). Caso base $DP[0][k] = 1$: la sottosequenza vuota ha prodotto $1 \leq k$.

$$DP[i][k] = \begin{cases} 1 & i = 0 \\ DP[i-1][k] & A[i] > k \\ DP[i-1][k] + DP[i-1][\lfloor k/A[i] \rfloor] & A[i] \leq k \end{cases}$$

Complessità. $O(nm)$.

countIncreasing: sottosequenze crescenti di dimensione k [22-07-13 · B3]

Traccia. Contare le sottosequenze **strettamente crescenti** di lunghezza esattamente k .

$$DP[i][j] = \begin{cases} 1 & j = 1 \\ \sum_{\substack{1 \leq m < i \\ A[m] < A[i]}} DP[m][j-1] & j > 1 \end{cases}$$

$DP[i][j]$ = sottosequenze lunghe j che **terminano** in i . **Risposta.** $\sum_{i=k}^n DP[i][k]$. **Complessità.** $O(kn^2)$.

B.5 DP su stringhe (famiglia LCS)

Tutte varianti dello schema LCS: se gli ultimi caratteri "coincidono" si arretra in diagonale sommando qualcosa, altrimenti si prende il massimo scartando un carattere dall'una o dall'altra stringa. Cambia solo cosa si somma e quale diagonale si usa.

LCS di tre stringhe [20-07-24 · B1]

Traccia. Lunghezza della più lunga sottosequenza comune a X, Y, Z .

$$DP[i][j][k] = \begin{cases} 0 & i = 0 \vee j = 0 \vee k = 0 \\ DP[i-1][j-1][k-1] + 1 & X[i] = Y[j] = Z[k] \\ \max\{DP[i-1][j][k], DP[i][j-1][k], DP[i][j][k-1]\} & \text{altrimenti} \end{cases}$$

Complessità. $\Theta(n_x n_y n_z)$. ⚠ Non è corretto fare $LCS(LCS(X, Y), Z)$ né min delle LCS a coppie.

Longest repeating subsequence [22-01-17 · B2]

Traccia. Più lunga sottosequenza **ripetuta** (presente due volte, senza condividere elementi).

$$DP[i][j] = \begin{cases} 0 & i = 0 \vee j = 0 \\ DP[i-1][j-1] + 1 & T[i] = T[j] \wedge i \neq j \\ \max\{DP[i-1][j], DP[i][j-1]\} & \text{altrimenti} \end{cases}$$

È $LCS(T, T)$ con il vincolo $i \neq j$. **Complessità.** $\Theta(n^2)$.

Connessioni non intersecanti = LCS [22-08-24 · B3]

Traccia. Massimo insieme di coppie (x, y) con $X[x] = Y[y]$, ciascun indice usato una volta e **senza intersezioni** (ordine preservato). È esattamente la LCS di X e Y .

$$DP[i][j] = \begin{cases} 0 & i = 0 \vee j = 0 \\ DP[i-1][j-1] + 1 & X[i] = Y[j] \\ \max\{DP[i-1][j], DP[i][j-1]\} & \text{altrimenti} \end{cases}$$

Complessità. $O(n^2)$.

Bottom-up.

```
LCS(X, Y, n, m):
  for i = 0 to n: DP[i][0] = 0          # una stringa vuota: LCS = 0
  for j = 0 to m: DP[0][j] = 0
  for i = 1 to n:
    for j = 1 to m:
      if X[i] == Y[j]:
        DP[i][j] = DP[i-1][j-1] + 1
      else:
        DP[i][j] = max(DP[i-1][j], DP[i][j-1])
  return DP[n][m]
```

Memoization.

```
LCS(X, Y, n, m):
  for i = 0 to n, j = 0 to m: DP[i][j] = NIL
  return solve(X, Y, n, m, DP)

solve(X, Y, i, j, DP):
  if i == 0 or j == 0: return 0
  if DP[i][j] != NIL: return DP[i][j]
  if X[i] == Y[j]:
    DP[i][j] = solve(X, Y, i-1, j-1, DP) + 1
  else:
    DP[i][j] = max(solve(X, Y, i-1, j, DP), solve(X, Y, i, j-1, DP))
  return DP[i][j]
```

MCS: sottosequenza comune di **valore** massimo [23-02-09 · B1]

Traccia. Su vettori di interi, massimizzare la **somma** dei valori di una sottosequenza comune.

$$DP[i][j] = \begin{cases} 0 & i \leq 0 \vee j \leq 0 \\ DP[i-1][j-1] + T[i] & T[i] = U[j] \wedge T[i] > 0 \\ \max\{DP[i-1][j], DP[i][j-1]\} & \text{altrimenti} \end{cases}$$

Al posto di +1 si somma $T[i]$ (solo se positivo). **Complessità.** $\Theta(nm)$.

maxL1: massima norma L_1 fra sottosequenze di uguale lunghezza [24-07-02 · B3]

Traccia. Selezionare due sottosequenze di ugual dimensione da V_1, V_2 massimizzando $\sum |V_1[\cdot] - V_2[\cdot]|$.

Idea. Come LCS, ma qui **ogni** coppia può essere accoppiata (non serve che i valori coincidano) e quando si accoppia si guadagna $|V_1[i] - V_2[j]|$. $DP[i][j]$ = miglior valore usando prefissi $V_1[1..i], V_2[1..j]$. Tre scelte: scartare $V_1[i]$ ($DP[i-1][j]$), scartare $V_2[j]$ ($DP[i][j-1]$), oppure **accoppiare** $V_1[i]$ con $V_2[j]$ e proseguire in diagonale ($DP[i-1][j-1] + |V_1[i] - V_2[j]|$). L'ugual lunghezza è garantita perché si aggiunge un elemento a ciascuna sottosequenza contemporaneamente.

$$DP[i][j] = \begin{cases} 0 & i = 0 \vee j = 0 \\ \max\{DP[i-1][j], DP[i][j-1], DP[i-1][j-1] + |V_1[i] - V_2[j]|\} & \text{altrimenti} \end{cases}$$

Complessità. $O(n_1 n_2)$.

LCNCS: sottosequenza comune **non consecutiva** [24-02-01 · B3]

Traccia. Come LCS, ma la sottosequenza non può contenere caratteri **contigui** nelle stringhe originali. Quando due caratteri coincidono conviene sempre prenderli, saltando entrambi i precedenti:

$$DP[i][j] = \begin{cases} 0 & i \leq 0 \vee j \leq 0 \\ DP[i-2][j-2] + 1 & T[i] = U[j] \\ \max\{DP[i-1][j], DP[i][j-1]\} & T[i] \neq U[j] \end{cases}$$

Complessità. $\Theta(nm)$.

Longest common substring (contigua) [20-09-04 · B2]

Traccia. Più lunga **sottostringa** (contigua) comune a due stringhe.

$$DP[i][j] = \begin{cases} 0 & i = 0 \vee j = 0 \vee P[i] \neq T[j] \\ DP[i-1][j-1] + 1 & P[i] = T[j] \end{cases}$$

La risposta è il **massimo sull'intera matrice**, non $DP[n][m]$. **Complessità.** $O(nm)$.

Misciotto 2: verificare se T è un mescolamento di S_1, S_2 [23-06-20 · B2]

Traccia. Dire se T (lunghezza $n_1 + n_2$) è ottenibile intercalando S_1 e S_2 mantenendone l'ordine (è lo *shuffle* del compito 16-01-26).

Idea. $B[i][j]$ = vero sse il prefisso $T[1..i+j]$ è uno shuffle di $S_1[1..i]$ e $S_2[1..j]$ (nota: l'indice in T è **determinato** da $i + j$, quindi bastano due indici, non tre). L'**ultimo** carattere di $T[i+j]$ deve provenire da S_1 (allora $S_1[i] = T[i+j]$ e il resto $B[i-1][j]$ dev'essere shuffle) **oppure** da S_2 (simmetrico). Base: due prefissi vuoti danno il prefisso vuoto di T .

$$B[i][j] = (S_1[i]=T[i+j] \wedge B[i-1][j]) \vee (S_2[j]=T[i+j] \wedge B[i][j-1]), \quad B[0][0] = \text{true}$$

Complessità. $O(n_1 n_2)$. 🚩 Le soluzioni "lineare a due indici" e " $\text{LCS}(S_1, T) + \text{LCS}(S_2, T) = n$ " sono **sbagliate**. (La versione "stampa tutti i misciotti" è in F.5, [23-06-20 · B1].)

B.6 DP tipo zaino / subset-sum

Zaino leggero: minimo sottoinsieme con somma C [21-02-08 · B2]

Traccia. Minimo numero di elementi di W la cui somma è esattamente C .

Idea. "Prendo / non prendo", ma invece di **contare** i sottoinsiemi si **minimizza la cardinalità**. $DP[i][c]$ = numero minimo di elementi dei primi i che sommano a c . Non prendo $W[i]$: $DP[i-1][c]$; prendo $W[i]$: $DP[i-1][c - W[i]] + 1$ (un elemento in più). Il $+\infty$ marca gli stati **impossibili** (target negativo, o positivo senza più elementi) così che il **min** li scarti automaticamente.

$$DP[i][c] = \begin{cases} 0 & c = 0 \\ +\infty & c < 0 \vee (c > 0 \wedge i = 0) \\ \min\{DP[i-1][c], DP[i-1][c - W[i]] + 1\} & c > 0 \wedge i > 0 \end{cases}$$

Complessità. $O(nC)$ (pseudopolinomiale).

Bottom-up.

```
zainoLeggero(W, n, C):                                # min #elementi con somma esatta C
    for c = 0 to C: DP[0][c] = +inf
    DP[0][0] = 0
    for i = 1 to n:
        for c = 0 to C:
            DP[i][c] = DP[i-1][c]                    # non prendo W[i]
            if W[i] <= c:
                DP[i][c] = min(DP[i][c], DP[i-1][c-W[i]] + 1) # prendo W[i]
    return DP[n][C]                                   # +inf => somma C irraggiungibile
```

Memoization.

```
zainoLeggero(W, n, C):
    for i = 0 to n, c = 0 to C: DP[i][c] = NIL
    return solve(W, n, C, DP)

solve(W, i, c, DP):
    if c == 0: return 0
    if i == 0: return +inf                        # somma > 0 senza elementi
    if DP[i][c] != NIL: return DP[i][c]
    DP[i][c] = solve(W, i-1, c, DP)              # non prendo W[i]
    if W[i] <= c:
        DP[i][c] = min(DP[i][c], solve(W, i-1, c-W[i], DP) + 1) # prendo W[i]
    return DP[i][c]
```

removeSum: massimo insieme rimovibile lasciando somma C [21-06-16 · B2]

Traccia. Dimensione del più grande insieme che può essere **rimosso** da A lasciando somma residua C . È il **duale** dello Zaino leggero: si trova il più piccolo sottoinsieme che vale C e si restituisce $n - |\text{esso}|$.

$$\text{removeSum}(A, n, C) = n - \text{smallSum}(A, n, C)$$

Complessità. $O(nC)$. (Se non c'è modo di ottenere C : $+\infty$ diventa $-\infty$.)

B.7 DP di selezione su sequenza con vincoli

Schema "prendo / non prendo l'elemento i ", spesso con uno **stato aggiuntivo** che ricorda il vincolo (quanti consecutivi, distanza, ultimo colore, numero di operazioni...). Comprende le varianti di **Kadane** (somma massimale), **house-robber** (Hateville) e **LIS pesata**.

Schema base — Hateville (house-robber). Massima somma di elementi di A senza sceglierne due adiacenti: $DP[i] = \max(DP[i-1], A[i] + DP[i-2])$, con $DP[0] = 0$ e $DP[1] = A[1]$.

Bottom-up.

```
hateville(A, n):                                     # max somma senza due elementi adiacenti
    DP[0] = 0
    DP[1] = A[1]
    for i = 2 to n:
```

```
DP[i] = max(DP[i-1], A[i] + DP[i-2])
return DP[n]
```

Memoization.

```
hateville(A, n):
    for i = 0 to n: DP[i] = NIL
    return solve(A, n, DP)

solve(A, i, DP):
    if i == 0: return 0
    if i == 1: return A[1]
    if DP[i] != NIL: return DP[i]
    DP[i] = max(solve(A, i-1, DP), A[i] + solve(A, i-2, DP))
    return DP[i]
```

maxSumEven: sottovettore di lunghezza pari a somma massima [20-01-14 · B2]

Traccia. Somma massima fra i sottovettori **contigui di lunghezza pari**.

$$DP[i] = \begin{cases} 0 & i \leq 1 \\ \max\{DP[i-2] + A[i-1] + A[i], 0\} & i > 1 \end{cases}$$

Variante di Kadane a passi di 2. **Risposta.** $\max_i DP[i]$. **Complessità.** $\Theta(n)$.

maxGain 2xn: sottoinsieme di caselle non adiacenti [20-06-12 · B1]

Traccia. Scacchiera $2 \times n$; massimizzare la somma di caselle a due a due non adiacenti (anche in diagonale). Basta prendere il **massimo di colonna** e applicare **Hateville** (house-robber) al vettore risultante — scegliere una casella esclude comunque l'intera colonna e le adiacenti. **Complessità.** $O(n)$.

Discordville: house-robber con finestra di esclusione variabile [22-02-07 · B2]

Traccia. Ogni casa i dona $D[i]$ ma non insieme alle $N[i]$ case precedenti. Massimizzare le donazioni.

Idea. House-robber con finestra di esclusione **variabile per casa**. $DP[i]$ = massimo raccolto considerando le prime i case. Due scelte: **salto** la casa i ($DP[i-1]$), oppure la **prendo**: allora devo saltare le $N[i]$ precedenti e ripartire da $DP[i-N[i]-1]$, cioè $D[i] + DP[i-N[i]-1]$. Se la finestra copre l'inizio dell'array ($i-N[i]-1 \leq 0$) prendere la casa vale semplicemente $D[i]$.

$$DP[i] = \begin{cases} D[1] & i = 1 \\ \max\{DP[i-1], D[i]\} & i > 1 \wedge i - N[i] - 1 \leq 0 \\ \max\{DP[i-1], D[i] + DP[i-N[i]-1]\} & i > 1 \wedge i - N[i] - 1 > 0 \end{cases}$$

Complessità. $\Theta(n)$.

Livelli bonus: salti di 1 o 2 [25-07-14 · B1]

Traccia. Dal livello 1 al livello n , da i si va a $i+1$ o $i+2$ raccogliendo $P[i]$; massimizzare il punteggio (si **deve** superare n).

$$DP[i] = P[i] + \max\{DP[i-1], DP[i-2]\}, \quad DP[1] = P[1], DP[2] = P[1] + P[2]$$

Risposta. $DP[n]$. **Complessità.** $\Theta(n)$, spazio $O(1)$.

ksequence: sottosequenza k -limitata di valore massimo [20-07-03 · B3]

Traccia. Massima somma di una sottosequenza in cui due valori consecutivi differiscono al più di k (LIS-like con $|A[i] - A[j]| \leq k$).

Idea. Schema LIS "pesata": $DP[i]$ = massima somma di una sottosequenza ammissibile che **termina** in i . Si attacca $A[i]$ alla migliore sottosequenza che finisce in un indice precedente $j < i$ **compatibile** ($|A[i] - A[j]| \leq k$); se nessun j va bene, la sottosequenza parte da sola (il $\{0\}$ nel $\max$). La risposta è il massimo su tutti i finali possibili.

$$DP[i] = A[i] + \max(\{0\} \cup \{DP[j] : 1 \leq j < i \wedge |A[i] - A[j]| \leq k\})$$

Risposta. $\max_i DP[i]$. **Complessità.** $O(n^2)$.

maxSumIncreasing: sottosequenza crescente di somma massima [21-06-16 · B3]

Traccia. Massimizzare la somma di una sottosequenza **strettamente crescente**.

$$DP[i] = A[i] + \max(\{0\} \cup \{DP[j] : j < i \wedge A[j] < A[i]\})$$

È la LIS "pesata" (si sostituisce $+1$ con $+A[i]$). **Complessità.** $O(n^2)$.

maxSumK: sottosequenza k -contigua massimale [21-07-26 · B3]

Traccia. Massimizzare la somma potendo **cancellare al più k elementi consecutivi** (duale del " $\leq k$ consecutivi presi" del 15-12-17).

Idea. Vincolo: gli elementi **scartati** non possono formare un blocco di più di k consecutivi. Serve dunque uno stato che ricordi quante cancellazioni consecutive restano: $DP[i][j]$ = miglior somma sui primi i elementi con ancora j cancellazioni consecutive disponibili. Per l'elemento i : o lo **prendo** (aggiungo $V[i]$ e **azzerò la corsa** di cancellazioni, cioè riparto col budget pieno k : $DP[i-1][k] + V[i]$), o lo **cancello** (solo se $j > 0$: consumo una cancellazione, $DP[i-1][j-1]$). Con $j = 0$ sono obbligato a prenderlo. La risposta $DP[n][k]$ parte con budget pieno.

$$DP[i][j] = \begin{cases} 0 & i = 0 \\ DP[i-1][k] + V[i] & j = 0 \text{ (costretto a prendere)} \\ \max\{DP[i-1][j-1], DP[i-1][k] + V[i]\} & j > 0 \end{cases}$$

j = cancellazioni consecutive ancora disponibili. **Risposta.** $DP[n][k]$. **Complessità.** $O(nk)$.

Stringhe eleganti: minime inversioni per ottenere 0^*1^* [23-08-23 · B3]

Traccia. Minimo numero di bit da invertire per rendere la stringa "elegante" (tutti gli 0 seguiti da tutti gli 1).

$$DP[i][b] = \begin{cases} 0 & i = 0 \\ DP[i-1][0] + S[i] & b = 0 \\ \min\{DP[i-1][0], DP[i-1][1]\} + (1 - S[i]) & b = 1 \end{cases}$$

b = valore dell' i -esimo bit dopo la trasformazione. **Risposta.** $\min\{DP[n][0], DP[n][1]\}$. **Complessità.** $O(n)$ (esiste anche una soluzione con **prefix-sum**).

Hateville PNRR: colorazione di case a costo minimo [26-02-02 · B3]

Traccia. Colorare n case con k colori, case adiacenti di colore diverso, minimizzando $\sum cost[h][colore(h)]$.

$$DP[h][c] = \begin{cases} cost[0][c] & h = 0 \\ cost[h][c] + \min_{p \neq c} DP[h-1][p] & h > 0 \end{cases}$$

Risposta. $\min_c DP[n-1][c]$. **Complessità.** $O(nk^2)$ (riducibile a $O(nk)$ tenendo minimo e secondo minimo di ogni riga).

Formazione stringhe: minime mosse (aggiungi carattere / duplica) [24-08-26 · B2]

Traccia. Minimo numero di mosse per costruire S , dove ogni mossa aggiunge un carattere in fondo **oppure** duplica l'intera stringa.

$$DP[i] = \begin{cases} 1 & i = 1 \\ \min\{DP[i-1] + 1, DP[i/2] + 1\} & i \text{ pari} \wedge S[1..i/2] = S[i/2+1..i] \\ DP[i-1] + 1 & \text{altrimenti} \end{cases}$$

Complessità. $O(n^2)$ (il test di duplicazione costa $O(n)$). Esiste anche un greedy che dal fondo dimezza appena possibile.

Borsa: k operazioni non sovrapposte a guadagno massimo [26-01-12 · B3]

Traccia. Eseguire **esattamente** k operazioni compra/vendi non sovrapposte massimizzando il guadagno.

Idea. $DP[i][\ell]$ = massimo guadagno usando i primi i giorni e completando **esattamente** ℓ operazioni. Si ragiona sull'ultima operazione: o il giorno i non è un giorno di **vendita** ($DP[i-1][\ell]$), oppure la ℓ -esima operazione **vende** al giorno i dopo aver comprato in un giorno $j < i$: guadagno $prices[i-1] - prices[j-1]$ più le $\ell-1$ operazioni fatte nei primi $j-1$ giorni ($DP[j-1][\ell-1]$), massimizzando su j . Il vincolo "esattamente k " è imposto dai casi base: con 0 operazioni il guadagno è 0; se i giorni non bastano ($i < 2\ell$, servono ≥ 2 giorni per operazione) lo stato è impossibile ($-\infty$).

$$DP[i][\ell] = \begin{cases} 0 & \ell = 0 \\ -\infty & \ell > 0 \wedge i < 2\ell \\ \max\left\{DP[i-1][\ell], \max_{1 \leq j \leq i-1} (DP[j-1][\ell-1] + prices[i-1] - prices[j-1])\right\} & \text{altrimenti} \end{cases}$$

Risposta. $DP[n][k]$. **Complessità.** $O(kn^2)$.

```
maxGain(prices, n, k):
    for i = 0 to n: DP[i][0] = 0                # 0 operazioni → guadagno 0
    for l = 1 to k: DP[0][l] = -inf            # 0 giorni, l>0 operazioni → impossibile
    for i = 1 to n, l = 1 to k:
        if i < 2*l:                            # troppi pochi giorni per l operazioni
            DP[i][l] = -inf
        else:
            DP[i][l] = DP[i-1][l]                # il giorno i non è di vendita
            for j = 1 to i-1:                    # provo ogni giorno di acquisto j
                DP[i][l] = max(DP[i][l], DP[j-1][l-1] + prices[i-1]-prices[j-1])
    return DP[n][k]
```

minSteps: minime mosse su griglia a "salti variabili" [25-02-03 · B3]

Traccia. Ogni cella $A[r][c]$ dice quanti passi al massimo si può saltare in basso o a destra; minimo numero di mosse da $(1, 1)$ a (n, n) .

$$DP[r][c] = \begin{cases} 0 & r = n \wedge c = n \\ 1 + \min\left(\min_{c < c' \leq \min(c+A[r][c], n)} DP[r][c'], \min_{r < r' \leq \min(r+A[r][c], n)} DP[r'][c]\right) & \text{altrimenti} \end{cases}$$

Il vincolo "solo destra/basso" rende i sottoproblemi un **DAG**: si risolve con memoization. **Complessità.** $O(n^3)$.

zeroUnbalance: massimo sbilanciamento 0/1 (riduzione a somma massimale) [20-02-04 · B2]

Traccia. Massimo, fra i sottovettori contigui, di $(\# \text{zeri} - \# \text{uni})$. Si costruisce B con $+1$ per ogni 0 e -1 per ogni 1: lo sbilanciamento è la **somma massimale** (Kadane) di B . **Complessità.** $\Theta(n)$. (Anche in G come riduzione a un algoritmo di lezione.)

B.8 DP di partizionamento / parentesizzazione

Massima espressività: parentesizzazione di $+$ e $\times$ [25-07-14 · B2]

Traccia. Massimo valore ottenibile parentesizzando $V_1 \text{ op}_1 V_2 \cdots \text{op}_{n-1} V_n$ con $\text{op}_i \in \{+, \times\}$.

Soluzione generale (matrix-chain). $DP[i][j]$ = massimo valore usando $V_i..V_j$:

$$DP[i][j] = \begin{cases} V_i & i = j \\ \max_{i \leq k < j} \text{apply}(DP[i][k], \text{op}_k, DP[k+1][j]) & i < j \end{cases}$$

Complessità. $\Theta(n^3)$. Con valori **tutti positivi** esiste anche un greedy $O(n)$: eseguire prima tutte le somme dei blocchi delimitati dai $\times$, poi moltiplicare i blocchi (perché $(x+y)z \geq x+yz$ per $x, y, z > 0$).

Bottom-up. ⚠ L'ordine di riempimento **non** è il solito doppio **for** i, j : si procede per **lunghezza crescente** dell'intervallo, altrimenti si leggono celle non ancora calcolate.

```
maxExpr(V, op, n):                                # max parentesizzando V1 op1 V2 ... opn-1
  Vn
  for i = 1 to n: DP[i][i] = V[i]                # intervallo di un solo valore
  for len = 2 to n:                               # ← lunghezza crescente dell'intervallo
    for i = 1 to n-len+1:
      j = i + len - 1
      DP[i][j] = -inf
      for k = i to j-1:                            # ultimo operatore applicato: op[k]
        v = apply(DP[i][k], op[k], DP[k+1][j])
        DP[i][j] = max(DP[i][j], v)
  return DP[1][n]
```

Memoization. La ricorsione trova l'ordine dei sottoproblemi da sola: non serve ragionare sulle lunghezze.

```
maxExpr(V, op, n):
  for i = 1 to n, j = 1 to n: DP[i][j] = NIL
  return solve(V, op, 1, n, DP)

solve(V, op, i, j, DP):
  if i == j: return V[i]                          # un solo valore
  if DP[i][j] != NIL: return DP[i][j]
  DP[i][j] = -inf
  for k = i to j-1:
    v = apply(solve(V, op, i, k, DP), op[k], solve(V, op, k+1, j, DP))
    DP[i][j] = max(DP[i][j], v)
  return DP[i][j]
```

C. Grafi

In questo periodo i grafi compaiono quasi sempre **travestiti**: come reti di flusso (D), come backtracking su cicli/cammini (F.5) o come DP su griglia/DAG (B.3, B.7). Resta un solo esercizio di **cammini minimi puri**. Le visite BFS/DFS "classiche" sono in Parte A (appendice B).

Scheletro tipico vs diverso. L'unico esercizio ha scheletro tipico: due esecuzioni dell'algoritmo di cammini minimi visto a lezione (Dijkstra / Bellman-Ford) più una scansione. Nessun codice dedicato.

C.1 Cammini minimi

Punto d'incontro a costo uguale [21-07-26 · B1]

Traccia. Grafo pesato orientato; due persone in s e t vogliono un nodo x raggiungibile con **lo stesso costo minimo** da entrambe. Dire se esiste.

Soluzione. Due esecuzioni di cammini minimi (una da s , una da t), ottenendo $d_s[\cdot]$ e $d_t[\cdot]$; si cerca un nodo con $d_s[u] = d_t[u] \neq \infty$.

$$\exists u \in V : d_s[u] = d_t[u] \neq \infty$$

Complessità. $O(m \log n)$ con Dijkstra (pesi ≥ 0); $O(mn)$ con Bellman-Ford se i pesi possono essere negativi. È l'analogo (semplificato) di *Alice, Bob e Carl* del 15-07-14.

D. Flusso massimo e matching

Il template da riprodurre a memoria. (1) Definisci i **nodi**: supersorgente s , superpozzo p , un livello di nodi per ogni "tipo" di entità. (2) Ogni vincolo del problema diventa una **capacità** su un arco. (3) Una soluzione valida esiste $\iff$ il flusso massimo raggiunge il **valore target** $|f^*|$. (4) Calcola $|V|$, $|E|$, $|f^*|$ e applica Ford-Fulkerson: $O(|f^*| (|V| + |E|))$. (5) Di' **dove si legge la soluzione**. In questo periodo il gadget più ricorrente è "**quota per categoria**" (§D.2): un livello intermedio di nodi che impone "almeno/esattamente x di tipo y ".

Scheletro tipico vs diverso. Qui la "soluzione scritta" è sempre la **costruzione della rete** (nodi, archi, capacità, valore di flusso target) + Ford-Fulkerson: è già riportata per *ogni* esercizio, quindi non manca nulla. Scheletro tipico = **assegnamento bipartito con capacità (D.1)**; gli scheletri "diversi" — comunque descritti per esteso — sono il **gadget quota per categoria (D.2)**, il **node splitting (D.3)** e il **ricalcolo / test di robustezza (D.4)**.

D.1 Assegnamento bipartito con capacità

Pulizie DISI: persone $\rightarrow$ uffici con tetto m [25-06-13 · B1]

Traccia. Ogni persona può pulire $\leq m$ uffici scelti fra quelli dichiarati; ogni ufficio va pulito **esattamente una volta**.

Costruzione della rete. Quattro livelli: la **supersorgente** s , un nodo per ogni **persona** $\ell \in L$, un nodo per ogni **ufficio** $o \in O$, il **superpozzo** t . Archi e capacità:

- $s \rightarrow \text{persona}_\ell$ con capacità m — così ogni persona può pulire **al più** m uffici;
- $\text{persona}_\ell \rightarrow \text{ufficio}_o$ con capacità 1, **solo** per gli uffici $o \in A_\ell$ che ℓ ha dichiarato — così si rispetta l'insieme ammissibile e ogni coppia persona-ufficio vale al più un'unità;
- $\text{ufficio}_o \rightarrow t$ con capacità 1 — così ogni ufficio è pulito **esattamente una volta**.

Perché funziona. Ogni unità di flusso è un assegnamento " ℓ pulisce o ": la capacità 1 verso t impedisce che un ufficio sia pulito due volte, la capacità m da s limita il carico di ogni persona, e la presenza degli archi solo su A_ℓ garantisce la compatibilità.

Risposta. Pianificazione valida $\iff |f^*| = |O|$ (tutti gli archi verso t saturi). **Complessità.** $|V| = n_L + n_O + 2$, $|E| = O(n_L n_O)$, flusso massimo $|f^*| \leq n_O$: con Ford-Fulkerson $O(|f^*|(|V| + |E|)) = O(n_L \cdot n_O^2)$.

Mensa: assegnamento per tre categorie indipendenti [25-08-25 · B1]

Traccia. Ogni studente può ricevere al più un primo, un secondo, un dessert fra i piatti che accetta; ogni piatto p ha m_p porzioni. Massimizzare le assegnazioni.

Costruzione della rete. L'osservazione chiave è che **le tre categorie sono indipendenti** (un primo non compete con un dessert), quindi si costruiscono **tre reti separate**, una per categoria $c \in \{\text{primo, secondo, dessert}\}$, e si sommano i flussi. Per la categoria c , con P_c i piatti di quel tipo: nodi s, t , un nodo per ogni **studente**, un nodo per ogni **piatto** $p \in P_c$. Archi:

- $s \rightarrow$ studente con capacità 1 — ogni studente riceve **al più un** piatto della categoria c ;
- studente $\rightarrow p$ con capacità 1, solo se $p \in A_{\text{studente}}$ (piatto gradito);
- $p \rightarrow t$ con capacità m_p — il numero di **porzioni** disponibili del piatto.

Perché la decomposizione. Se si fondessero le tre categorie in un'unica rete, la capacità 1 da s non riuscirebbe a impedire, ad esempio, di dare due primi e nessun secondo allo stesso studente. Separando le reti, quella capacità vale *per categoria* e ciascuno riceve al più un piatto per tipo.

Risposta. $F_{\text{tot}} = F_{\text{primo}} + F_{\text{secondo}} + F_{\text{dessert}}$ = numero massimo di assegnazioni studente-piatto. **Complessità.** Somma delle tre istanze, ciascuna $O((V_c + E_c) \cdot n)$ con $n = |S|$.

Giardinaggio: piante $\rightarrow$ vasi [25-02-03 · B2]

Traccia. La pianta i entra nel vaso j se $P[i] \leq V[j]$; ogni vaso una pianta. Massimizzare le piante invase.

Costruzione della rete (alternativa). È un matching bipartito con capacità: nodi s, t , un nodo per ogni **pianta**, un nodo per ogni **vaso**. Archi: $s \rightarrow \text{pianta}_i$ (cap. 1); $\text{pianta}_i \rightarrow \text{vaso}_j$ (cap. 1) **solo se** $P[i] \leq V[j]$; $\text{vaso}_j \rightarrow t$ (cap. 1, un vaso ospita una sola pianta). Il flusso massimo è il numero massimo di piante invase.

Ma: la soluzione **ufficiale** è **greedy** e molto più efficiente ($O(n \log n + m \log m)$): si ordinano piante e vasi e si assegna il vaso più piccolo capiente alla pianta più piccola — vedi E.1.

D.2 Reti con gadget: quote per categoria e diversità

Quando serve "almeno/esattamente x elementi **di un certo tipo**" (esperti, senior, un genere), si sdoppia il nodo-entità in nodi-gadget per tipo, e la capacità sugli archi del gadget impone la quota. Compare in **8 esami** su 33.

Commissioni di laurea: ≥ 2 esperti su 5 [24-01-09 · B1]

Traccia. k commissioni da 5 professori, di cui **almeno 2 esperti** dell'area; ogni professore in al più una commissione.

Costruzione della rete. Nodi: s ; un nodo per ogni **professore** $p_1 \dots p_n$; **due** nodi-gadget per ogni commissione i — c_i^2 (raccolge i 2 esperti) e c_i^3 (raccolge gli altri 3, chiunque); il pozzo t . Archi e capacità:

- $s \rightarrow \text{prof}$ con capacità 1 — un professore in **al più una** commissione;
- $\text{prof} \rightarrow c_i^2$ con capacità 1, **solo** se quel professore è esperto dell'area della commissione i ;
- $\text{prof} \rightarrow c_i^3$ con capacità 1, per **ogni** professore e **ogni** commissione (chiunque può occupare uno dei 3 posti liberi);
- $c_i^2 \rightarrow t$ con capacità 2 e $c_i^3 \rightarrow t$ con capacità 3.

Perché funziona (il gadget "quota"). Sdoppiando la commissione in c_i^2/c_i^3 si impone la quota: il taglio $c_i^2 \rightarrow t = 2$ forza **esattamente 2** posti riempibili solo da esperti (gli unici collegati a c_i^2), mentre $c_i^3 \rightarrow t = 3$ riempie i restanti con chiunque. Un professore, entrando con una sola unità da s , finisce in un solo posto di una sola commissione.

Risposta. Fattibile $\iff |f^*| = 5k$. **Complessità.** $|V| = 2 + n + 2k$, $|E| = 2n + nk + 2k$; il flusso è limitato da $\min(n, 5k) = 5k$ (serve $n \geq 5k$), quindi $O(5k(|V| + |E|)) = O(nk^2)$.

Facoltadi: quota di genere per squadra [23-02-09 · B2]

Traccia. 14 squadre da 21, ciascuna con **almeno 7 del genere meno rappresentato**; ogni studente nel suo dipartimento o in quello affine.

Costruzione della rete. Nodi: s ; un nodo per ogni **studente**; **tre** nodi-gadget per squadra j ($1 \leq j \leq 14$) — s_j^M (raccolge 7 maschi), s_j^F (7 femmine), s_j^X (7 di **qualunque** genere, incluso "preferisco non rispondere"); il pozzo t . Archi:

- $s \rightarrow$ studente con capacità 1 — ogni studente in una sola squadra;
- ogni **maschio** $i \rightarrow s_{d(i)}^M, s_{d'(i)}^M$ (cap. 1), verso il suo dipartimento e quello affine; ogni **femmina** $i \rightarrow s_{d(i)}^F, s_{d'(i)}^F$; ogni studente (senza distinzione) $\rightarrow s_{d(i)}^X, s_{d'(i)}^X$ — in totale $\leq 4n$ archi;
- $s_j^M, s_j^F, s_j^X \rightarrow t$ ciascuno con capacità 7.

Perché funziona. Ogni squadra è satura solo se raccoglie 7 maschi (s_j^M), 7 femmine (s_j^F) e 7 studenti qualsiasi (s_j^X): $7 + 7 + 7 = 21$ con la garanzia di **almeno 7 per ciascun genere**. La capacità 1 da s impedisce a uno studente di occupare più di un posto/categoria.

Risposta. Fattibile $\iff |f^*| = 14 \cdot 21 = 294$. **Complessità.** $|V| = n + 44$, $|E| \leq 5n + 42$; flusso limitato da 294 (costante) $\Rightarrow O(n)$.

Conferenza: 3 reviewer per paper, ≥ 2 esperti [23-06-20 · B3]

Traccia. Ogni paper vuole 3 reviewer, di cui ≥ 2 esperti dell'area; ogni reviewer $\leq k$ review.

Costruzione della rete (tre nodi per paper). Nodi: s ; un nodo per ogni **reviewer** r_i ; **tre** nodi-gadget per ogni paper j — p_j^{ne} (raccolge i non esperti), p_j^{ex} (raccolge gli esperti), p_j^{tot} (impone il totale di 3); il pozzo t . Archi:

- $s \rightarrow r_i$ con capacità k — ogni reviewer fa **al più** k review;
- $r_i \rightarrow p_j^{ex}$ (cap. 1) **se** r_i è esperto nell'area del paper j , **altrimenti** $r_i \rightarrow p_j^{ne}$ (cap. 1): **un solo arco** per coppia reviewer-paper;
- $p_j^{ne} \rightarrow p_j^{tot}$ con capacità 1 (al più **un** non esperto per paper) e $p_j^{ex} \rightarrow p_j^{tot}$ con capacità 3;
- $p_j^{tot} \rightarrow t$ con capacità 3 (totale di 3 reviewer).

Perché servono tre nodi. Il singolo arco reviewer $\rightarrow$ paper evita la **doppia review** (un reviewer non può contare sia come esperto sia come non esperto sullo stesso paper). Il tappo $p_j^{ne} \rightarrow p_j^{tot} = 1$ ammette **al massimo 1 non esperto**, quindi degli altri 2 sono per forza esperti (≥ 2 esperti). Il nodo p_j^{tot} con arco 3 verso t serve a **fissare a 3** il totale: senza di esso, alzare a 3 la capacità $p_j^{ex} \rightarrow t$ permetterebbe fino a 4 review, mentre la variante ingenua a due nodi (p_j^{ex} cap. 2 verso t) **obbligherebbe** esattamente 2 esperti, fallendo quando *tutti* i reviewer di un paper sono esperti.

Risposta. Fattibile $\iff |f^*| = 3n_p$. **Complessità.** $|V| = 2 + n_r + 3n_p$, $|E| = n_r + 2n_p + n_r n_p$; flusso $\leq 3n_p \Rightarrow O(n_p^2 n_r)$.

Junior vs senior: quota di senior per servizio [26-01-12 · B2]

Traccia. Ogni servizio j vuole **esattamente** vt_j volontari, di cui **almeno** vs_j senior.

Costruzione della rete. Nodi: s, t ; un nodo vJ_i per ogni **volontario junior** e vs_i per ogni **senior**; per ogni servizio j **due** nodi-gadget s_j^J (junior assegnati) e s_j^S (senior assegnati) più un nodo s_j che raccoglie il servizio. Archi:

- $s \rightarrow vJ_i$ e $s \rightarrow vs_i$ con capacità 1 — ogni volontario a **un solo** servizio;
- $vJ_i \rightarrow s_j^J$ (cap. 1) per ogni servizio $j \in \text{avail}_i$ disponibile; $vs_i \rightarrow s_j^S$ (cap. 1) idem;
- $s_j^J \rightarrow s_j$ con capacità $vt_j - vs_j$ (**tetto ai junior**) e $s_j^S \rightarrow s_j$ con capacità vs_j ;

- $s_j \rightarrow t$ con capacità vt_j .

Perché funziona. Separando junior e senior in due nodi-gadget, il tappo $s_j^J \rightarrow s_j = vt_j - vs_j$ impone che **al più** $vt_j - vs_j$ dei vt_j posti siano junior; di conseguenza, per saturare $s_j \rightarrow t = vt_j$, **almeno** vs_j assegnati devono essere senior. Ogni volontario, entrando con una sola unità da s , non può stare in più servizi.

Risposta. Fattibile $\iff |f^*| = \sum_j vt_j$ (tutti gli archi $s_j \rightarrow t$ saturi). **Complessità.** $O(n^2m)$.

Personale ospedaliero: infermieri/OSS per reparto [26-02-02 · B2]

Traccia. Ogni reparto j vuole **esattamente** m_j operatori, con $\geq m_{ij}$ infermieri e $\geq m_{oj}$ OSS.

Osservazione preliminare. Se $\sum_j m_j \neq n$ non esiste assegnazione (ogni operatore va collocato ed ogni reparto ne vuole esattamente m_j): è la **precondizione** di fattibilità.

Costruzione della rete. Nodi: s, t ; un nodo i_h per ogni **infermiere** e o_h per ogni **OSS**; per ogni reparto j **due** nodi-gadget r_j^I (infermieri assegnati) e r_j^O (OSS assegnati) più un nodo r_j . Archi:

- $s \rightarrow i_h$ e $s \rightarrow o_h$ con capacità 1 — ogni operatore a **un solo** reparto;
- $i_h \rightarrow r_j^I$ (cap. 1) e $o_h \rightarrow r_j^O$ (cap. 1) **solo** se il reparto è in capable_h (qualifica rispettata);
- $r_j^I \rightarrow r_j$ con capacità $m_j - m_{oj}$ e $r_j^O \rightarrow r_j$ con capacità $m_j - m_{ij}$ (**tappi simmetrici**);
- $r_j \rightarrow t$ con capacità m_j .

Perché funziona. Struttura identica a Junior/senior ma con **entrambe** le quote: cappando gli infermieri a $m_j - m_{oj}$ si lascia spazio ad almeno m_{oj} OSS, e cappando gli OSS a $m_j - m_{ij}$ si garantiscono almeno m_{ij} infermieri; l'arco $r_j \rightarrow t = m_j$ fissa il totale a m_j .

Risposta. Fattibile $\iff |f^*| = n = \sum_j m_j$. **Complessità.** $O(n^2m)$.

Piano strategico: budget dei gruppi, quota K ai delegati [22-06-21 · B2]

Traccia. Finanziare ogni azione con budget x_i dai gruppi cui è associata; ogni delegato deve ricevere esattamente K budget.

Costruzione della rete. Il budget "scorre" come flusso da s ai delegati. Nodi: s, t ; un nodo G_j per ogni **gruppo**; una coppia $A_i \rightarrow A'_i$ per ogni **azione** (il gadget di quota); un nodo D_ℓ per ogni **delegato**. Archi:

- $s \rightarrow G_j$ con capacità b_j — ogni gruppo eroga al più il suo **budget** b_j ;
- $G_j \rightarrow A_i$ per $j \in \text{grp}_i$ (azione associata al gruppo), capacità $+\infty$ (di fatto limitata da b_j);
- $A_i \rightarrow A'_i$ con capacità x_i — il **gadget** che forza l'azione a ricevere **esattamente** x_i quando l'arco è saturo;
- $A'_i \rightarrow D_\ell$ per $\ell \in \text{del}_i$, capacità $+\infty$ (limitata da K);
- $D_\ell \rightarrow t$ con capacità K — ogni delegato riceve esattamente K .

Perché funziona. L'unità di flusso è un'unità di budget. Lo **strozzamento** $A_i \rightarrow A'_i = x_i$ garantisce che, se saturo, l'azione abbia raccolto esattamente il budget richiesto dai suoi gruppi; l'arco $D_\ell \rightarrow t = K$ impone la quota per delegato.

Risposta. Fattibile $\iff$ **tutti** gli archi (A_i, A'_i) sono saturi, cioè $|f^*| = X = \sum_i x_i$. **Complessità.** $|V| = O(k + n + d)$, $|E| = O((k + d)n)$; $O((k + d)nX)$.

Programmazione esami: corsi $\rightarrow$ (aula, orario) $\rightarrow$ supervisor [22-08-24 · B1]

Traccia. Assegnare a ogni corso un'aula (capiente a sufficienza), uno slot e un supervisore (disponibile in quello slot, $\leq d$ esami).

Costruzione della rete. Quattro livelli fra s e t : un nodo per ogni **corso** c_i ; un nodo per ogni **coppia (aula, orario)** (a_j, o_k) — è qui che aula e slot vengono fusi in un'unica risorsa; un nodo per ogni **supervisore** s_ℓ . Archi:

- $s \rightarrow c_i$ con capacità 1 — ogni corso una sola volta;
- $c_i \rightarrow (a_j, o_k)$ con capacità 1, **solo** se l'aula a_j è **abbastanza capiente** per il corso c_i ;
- $(a_j, o_k) \rightarrow s_\ell$ con capacità 1, **solo** se lo slot $o_k \in T_\ell$ è fra quelli in cui s_ℓ è disponibile;
- $s_\ell \rightarrow t$ con capacità d — ogni supervisore segue **al più** d esami.

Perché funziona. La capacità 1 sulla coppia (a_j, o_k) (verso il supervisore) fa sì che quella aula-slot sia usata da **un solo** corso; la 1 da s fa sì che ogni corso ottenga al più una tripla (aula, slot, supervisore) valida; il tetto d verso t limita il carico dei supervisori.

Complessità. $|V| = C + AO + S + 2$, $|E| = O(AO(C + S))$; flusso $\leq C \Rightarrow O(AOC(C + S))$.

Didattica pandemia: corsi in aule $\times$ slot, quota per triennale [20-07-24 · B2]

Traccia. Allocare corsi in coppie (aula, slot); ogni corso ≤ 2 slot, ogni triennale $\leq k$ slot.

Costruzione della rete. Cinque livelli: un nodo per ogni triennale t_k ; un nodo per ogni corso c_i ; un nodo per ogni coppia (aula, slot) (a, s) (con n aule e 25 slot, sono $25n$ nodi). Archi:

- $s \rightarrow t_k$ con capacità k — ogni triennale usa **al più** k slot;
- $t_k \rightarrow c_i$ con capacità 2, per i corsi con $t[i] = k$ — ogni corso occupa **al più 2** slot;
- $c_i \rightarrow (a, s)$ con capacità 1, per le aule $a \in A_i$ e gli slot $s \in S_i$ ammissibili;
- $(a, s) \rightarrow t$ con capacità 1 — ogni coppia aula-slot assegnata a **un solo** corso.

Perché funziona. Le capacità propagano i tre vincoli in cascata: 1 verso t (una coppia aula-slot per un corso), 2 sull'arco triennale $\rightarrow$ corso (due slot per corso), k da s (budget di slot della triennale). Il flusso massimo è il numero massimo di slot allocati, limitato da $2m$.

Complessità. $|V| = m + 25n + t + 2$, $|E| = O(mn)$; flusso $\leq 2m \Rightarrow O(m(mn + t))$.

SWERC: diversità di paese al tavolo [23-07-11 · B1]

Traccia. Assegnare studenti a tavoli; ogni tavolo $\leq k$ studenti e ≤ 3 dal medesimo paese.

Costruzione della rete. Nodi: s ; un nodo per ogni università U_i ; un nodo-gadget per ogni coppia (tavolo, paese) (T_j, p) (con n_t tavoli e i paesi, $6n_t$ nodi); un nodo per ogni tavolo T_j ; il pozzo t . Archi:

- $s \rightarrow U_i$ con capacità n_i — numero di studenti di quell'università da collocare;
- $U_i \rightarrow (T_j, P(U_i))$ con capacità 1 — l'università manda i suoi studenti **solo** verso le coppie del **proprio paese** $P(U_i)$, e **al più uno** per tavolo;
- $(T_j, p) \rightarrow T_j$ con capacità 3 — al più **3 studenti dello stesso paese** p per tavolo;
- $T_j \rightarrow t$ con capacità k — al più k studenti per tavolo.

Perché serve il gadget (tavolo, paese). Fa un doppio lavoro: la capacità 1 in ingresso dall'università impedisce a una stessa università di piazzare più di uno studente su un dato tavolo; la capacità 3 in uscita verso il tavolo limita a 3 i partecipanti di ogni paese. Mettendo solo i paesi tra università e tavoli si **perderebbe** il vincolo "un solo studente per università per tavolo".

Risposta. Assegnamento massimo = $|f^*|$ (fino a N studenti). **Complessità.** $|V| = 2 + n_u + 7n_t$, $|E| = O(n_t n_u)$; flusso $\leq N \Rightarrow O(N n_t n_u)$.

D.3 Node splitting e cammini disgiunti

Vie di fuga: cammini **nodo**-disgiunti su griglia [24-07-02 · B1]

Traccia. N studenti su una griglia devono raggiungere il bordo con percorsi **senza posizioni in comune**.

Costruzione della rete (node splitting). Il flusso massimo conta cammini **arco**-disgiunti; per imporre che siano **nodo**-disgiunti si **sdoppia ogni cella**. Nodi: s, t ; per ogni posizione (r, c) una coppia $in_{r,c} \rightarrow out_{r,c}$. Archi e capacità (tutte 1):

- $in_{r,c} \rightarrow out_{r,c}$ con capacità 1 — l'**arco-strozzatura** che limita a **uno** il flusso attraverso quella posizione;
- $out_{r,c} \rightarrow in_{r',c'}$ verso le 4 celle **adiacenti** (su/giù/sinistra/destra, quando esistono) — le mosse di spostamento;
- $s \rightarrow in_{r_i,c_i}$ per la cella di partenza di ogni **studente** i ;
- $out_{r,c} \rightarrow t$ per ogni cella di **bordo** (uscita dalla griglia).

Perché funziona. Poiché tutta la "portata" di una cella passa dall'unico arco $in \rightarrow out$ di capacità 1, due cammini non possono condividere una posizione: il node splitting trasforma il vincolo sui **nodi** in un vincolo sugli **archi**, gestibile da Ford-Fulkerson.

Risposta. $true \iff |f^*| = N$ (tutti gli studenti raggiungono il bordo). **Complessità.** $|V| = 2RC + 2, |E| = O(RC + N)$; $flusso \leq N \Rightarrow O((RC + N) \cdot N)$.

D.4 Test di robustezza / ricalcolo

Rete idrica: quale tubo, se chiuso, riduce il servizio? [23-08-23 · B1]

Traccia. Rete con sorgenti (producono $in(v)$) e consumatori (richiedono $out(v)$). Dire se **chiudere anche una sola** delle tubazioni guaste E_{per} riduce il servizio a qualche consumatore.

Costruzione della rete. Si parte dal grafo fisico $G = (V, E)$ e si aggiungono super-sorgente e super-pozzo: $V' = V \cup \{s, t\}$. Archi:

- gli **archi originali** $(u, v) \in E$ con la loro **portata** $c'(u, v) = c(u, v)$;
- $s \rightarrow v$ per ogni nodo **sorgente** $v \in V_{in}$, con capacità $c'(s, v) = in(v)$ (quanto quel nodo produce);
- $v \rightarrow t$ per ogni **consumatore** $v \in V_{out}$, con capacità $c'(v, t) = out(v)$ (quanto quel nodo richiede).

Procedura (test di robustezza). Si calcola il flusso massimo di riferimento. Poi **per ogni** tubazione a rischio $(x, y) \in E_{per}$ si **ripete** il calcolo con quella capacità temporaneamente **azzerata** e si confronta col riferimento: se una qualunque chiusura abbassa il flusso, il servizio a qualche consumatore si riduce.

$$T(n) = |E_{per}| \cdot tot \cdot (|V| + |E|), \quad tot = \sum_{v \in V_{out}} out(v)$$

dove tot (richiesta complessiva) limita il flusso massimo, e il fattore $|E_{per}|$ conta i ricalcoli. Stessa filosofia del *ricalcolo incrementale del flusso* del 15-12-17.

E. Greedy

Da fare sempre: o si **dimostra** l'ottimalità (quasi sempre con l'**argomento di scambio**), oppure si esibisce un **controesempio** piccolo e verificabile a mano. In questo periodo il docente ama chiedere di **smontare** greedy plausibili (§E.2): quando la traccia elenca due o tre strategie greedy "trovate su Internet", quasi sempre almeno una è sbagliata.

Scheletro tipico vs diverso. Qui la "soluzione scritta" è la **regola greedy + la sua dimostrazione** (o, in negativo, il **controesempio**): è già presente per ogni esercizio. Scheletro tipico = **ordinamento + argomento di scambio** (E.1); scheletro "diverso" = i greedy **da smontare con un controesempio** (E.2).

E.1 Ordinamento e argomento di scambio

findOrder: permutazione che rispetta un pattern di $<$ e $>$ [21-02-08 · B3]

Traccia. Data A e una sequenza di segni $</>$, produrre una permutazione di A che li rispetti.

Soluzione greedy. Ordinare A ; scorrere i segni prendendo il **minimo** rimanente se il segno è $<$, il **massimo** se è $>$.

Complessità. $O(n \log n)$, dominata dall'ordinamento.

Correttezza (induzione su n). Si dimostra contemporaneamente che una soluzione esiste sempre e che il greedy ne costruisce una.

- *Base* $n = 1$: un solo valore, nessun segno — la sequenza è banalmente valida.
- *Passo*: con A ordinato, sia s_1 il primo segno. Se $s_1 = <$ scegliamo il minimo $a_{\min}$ e ci riduciamo al problema con i restanti $n - 1$ valori e gli $n - 2$ segni successivi; per ipotesi induttiva esso ammette una soluzione S' . Poiché $a_{\min}$ è il minimo di tutti, $a_{\min} \leq S'[1]$, quindi la concatenazione $a_{\min} < S'$ rispetta anche s_1 . Simmetrico per $s_1 = >$, scegliendo il massimo (che è $\geq S'[1]$).

Il greedy segue passo per passo questa induzione, dunque produce sempre una permutazione valida. ■

Giardinaggio: piante $\rightarrow$ vasi (matching greedy) [25-02-03 · B2]

Traccia. Massimizzare le piante invase ($P[i] \leq V[j]$). **Soluzione.** Ordinare P e V crescenti, scorrere con due indici: se $P[i] \leq V[j]$ si invasa (avanzano entrambi i e j), altrimenti il vaso j è troppo piccolo per **ogni** pianta rimasta e si scarta ($j++$). **Complessità.** $O(n \log n + m \log m)$ (dominata dagli ordinamenti; lo scorrimento è $O(n + m)$). Molto più veloce del flusso.

Correttezza (argomento di scambio). Ordinati P, V , siano $p_{\min} = P[1]$ la pianta minima e v^* il **primo** vaso con $V[v^*] \geq p_{\min}$ (se non esiste, nessuna pianta è invasabile e il greedy è ottimo, con 0 assegnamenti). Nota che *ogni* vaso capace di contenere $p_{\min}$ ha capacità $\geq V[v^*]$, perché v^* è il minimo capace. Data una soluzione ottima, la si trasforma in una altrettanto grande che fa la scelta greedy ($p_{\min} \rightarrow v^*$):

- se l'ottimo invasa $p_{\min}$ in un vaso $w \neq v^*$ e v^* è **libero**: sposto $p_{\min}$ in v^* (valido, $V[v^*] \geq p_{\min}$) — invase invariate;
- se v^* contiene un'altra pianta p' e $p_{\min}$ è in w : **scambio** $p_{\min} \rightarrow v^*$ e $p' \rightarrow w$, valido perché $p' \leq V[v^*] \leq V[w]$ — invariate;
- se $p_{\min}$ non è invasa: v^* deve essere occupato da qualche p' (fosse libero, aggiungerei $p_{\min}$ guadagnando $+1$, contro l'ottimalità); rimpiazzo p' con $p_{\min}$ in v^* — invariate.

In ogni caso esiste un ottimo che invasa $p_{\min}$ in v^* ; rimossi pianta e vaso, si itera sul sottoproblema identico. ■

lpp: massima permutazione palindroma delle cifre [20-07-03 · B1]

Traccia. Riordinare le cifre di un numero per ottenere la **più grande** permutazione palindroma.

Soluzione greedy. Contare le cifre (stile counting sort, $O(n)$), poi piazzarle **dall'esterno verso l'interno** partendo dalla più alta (a coppie), con la cifra dispari eventuale al centro. **Complessità.** $O(n)$.

Correttezza (scambio per posizione). Premessa: una permutazione palindroma esiste $\iff$ **al più una** cifra ha conteggio dispari (quella eventuale finisce forzatamente al centro, posizione meno significativa). Un palindromo è determinato dalla sua **metà sinistra**, e il valore del numero è dominato dalle posizioni più significative (a sinistra). Sia *OPT* il palindromo massimo: se in una posizione i della metà sinistra avesse una cifra d mentre una cifra $d' > d$ occupasse una posizione $j > i$ (più a destra, meno significativa) della stessa metà, scambiare $d \leftrightarrow d'$ **con le rispettive immagini speculari** preserva la palindromicità e **aumenta** il numero (cresce una cifra più significativa) — contraddizione con l'ottimalità. Quindi in *OPT* la metà sinistra è non crescente da sinistra a destra: esattamente ciò che costruisce il greedy piazzando ogni volta la cifra disponibile più alta verso l'esterno. ■

longestChain: catena massima di coppie = interval scheduling [20-01-14 · B1]

Traccia. Coppie $(X[i], Y[i])$ con $X[i] < Y[i]$; catena = sequenza con $Y[i_k] \leq X[i_{k+1}]$. Trovare la catena più lunga.

Soluzione. È esattamente l'**Insieme Indipendente di Intervalli** (activity selection): si richiama `independentSet()` — greedy per **tempo di fine crescente**. **Complessità.** $\Theta(n \log n)$, dominata dall'ordinamento.

Correttezza (il greedy "sta avanti"). Ogni coppia è un intervallo $[X_i, Y_i]$; una catena è un insieme di intervalli a due a due disgiunti (con $Y_{i_k} \leq X_{i_{k+1}}$), quindi la catena più lunga è l'insieme indipendente massimo. Il greedy prende ripetutamente l'intervallo ammissibile con **fine Y minima**. Siano $g_1, \dots, g_k$ le scelte greedy e $o_1, \dots, o_m$ un ottimo, entrambi ordinati per tempo di fine. Per induzione $Y_{g_r} \leq Y_{o_r}$ per ogni r : vale per $r = 1$ (il greedy prende il fine minimo assoluto); se vale per $r - 1$, allora o_r inizia dopo $Y_{o_{r-1}} \geq Y_{g_{r-1}}$, quindi o_r era ammissibile anche per il greedy, che però ha scelto il fine minimo — da cui $Y_{g_r} \leq Y_{o_r}$. Se fosse $m > k$, l'intervallo o_{k+1} inizierebbe dopo $Y_{o_k} \geq Y_{g_k}$: sarebbe ancora ammissibile e il greedy non si sarebbe fermato a k , assurdo. Dunque $k = m$ e il greedy è ottimo. ■

minRemove: minime rimozioni perché $\max - \min \leq k$ [21-01-18 · B2]

Traccia. Minimo numero di elementi distinti da rimuovere da A perché nel resto $\max - \min \leq k$.

Soluzione (ottima, $O(n \log n)$). Ordinare; con **due indici** (finestra scorrevole) si trova il più lungo sottovettore con $A[j] - A[i] \leq k$ e si restituisce $n - (\text{sua lunghezza})$. Esistono anche una DP a intervalli $O(n^2)$ ($DP[i][j] = 0$ se $A[j] - A[i] \leq k$, altrimenti $\min\{DP[i+1][j], DP[i][j-1]\}$) e una variante con ricerca binaria. (Catalogata anche in G.)

Correttezza. Ordinato A , rimuovere il minimo equivale a **conservare** il massimo insieme S con $\max S - \min S \leq k$. (1) *Un S ottimo è un tratto contiguo dell'array ordinato:* fissati $\min S = A[i]$ e $\max S = A[j]$, ogni $A[h]$ con $i < h < j$ ha $A[i] \leq A[h] \leq A[j]$, quindi aggiungerlo **non cambia lo spread** e non può che aumentare $|S|$ — conviene tenerli tutti. Perciò ogni soluzione candidata è un sottovettore $A[i..j]$ con $A[j] - A[i] \leq k$, e minimizzare le rimozioni = massimizzarne la lunghezza. (2) *La finestra a due indici trova il massimo:* essendo lo spread monotono crescente nell'ordinato, se $A[j] - A[i] > k$ allora nessun $j' > j$ va bene con lo stesso i (spread solo maggiore), e conviene avanzare i ; se $A[j] - A[i] \leq k$ si registra la lunghezza e si estende j . Poiché i e j crescono soltanto, la scansione è $O(n)$ e per ogni i considera la finestra massima. ■

E.2 Greedy che NON funziona

Tape: file su nastro, due strategie sbagliate [23-01-17 · B2]

Traccia. Minimizzare il costo medio di lettura $\frac{1}{n} \sum_i c_i p_i$ (variante di *file su nastro*). Si valutino: (a) ordine per **probabilità decrescente**; (b) ordine per **lunghezza crescente**.

Controesempio (entrambe sbagliate, $n = 2$). Il costo di leggere il file in posizione t è $p_t \cdot$ (lunghezza cumulata fino a t inclusa); si minimizza la media $\frac{1}{n} \sum_t p_t \ell_t^{\text{cum}}$. Bastano due file:

- *Prob. decrescente* — $p = [0.6, 0.4]$, $d = [200, 100]$ (già ordinati): costo $\frac{0.6 \cdot 200 + 0.4 \cdot (200 + 100)}{2} = \frac{120 + 120}{2} = 120$; l'ordine **inverso** dà $\frac{0.4 \cdot 100 + 0.6 \cdot (100 + 200)}{2} = \frac{40 + 180}{2} = 110 < 120$. ×
- *Lunghezza crescente* — $p = [0.1, 0.9]$, $d = [100, 200]$ (già crescenti): costo $\frac{0.1 \cdot 100 + 0.9 \cdot (100 + 200)}{2} = \frac{10 + 270}{2} = 140$; l'inverso dà $\frac{0.9 \cdot 200 + 0.1 \cdot (200 + 100)}{2} = \frac{180 + 30}{2} = 105 < 140$. ×

L'ottimo (non richiesto) ordina per **rapporto ℓ_i/p_i crescente**, dimostrabile con argomento di scambio su due file adiacenti — vedi la raccolta 2014–2019, [14-06-16 · Es.3].

Multi core: scheduling di task, tre strategie [24-06-14 · B2]

Traccia. Assegnare n task a k processori minimizzando il tempo di completamento (makespan). Si valutino: (a) ordine di input; (b) lunghezza **crescente**; (c) lunghezza **decrescente** — ciascuno assegnando al processore attualmente più scarico.

Controesempio (tutte e tre non ottime). Makespan = massimo carico fra i processori; ogni task va al processore attualmente più scarico (a parità, il primo).

- *Crescente* — $A = [1, 1, 2]$, $k = 2$: il greedy assegna $1 \rightarrow P_1$, $1 \rightarrow P_2$, $2 \rightarrow P_1$, carichi $(3, 1)$, makespan **3**; l'ottimo $\{1, 1\} \mid \{2\}$ dà **2**. × (La soluzione ufficiale scrive 4 per una svista aritmetica; il greedy least-loaded dà comunque $3 > 2$, quindi il controesempio regge.)
- *Decrescente (LPT)* — $A = [3, 3, 2, 2, 2]$, $k = 2$: greedy $3 \rightarrow P_1$, $3 \rightarrow P_2$, $2 \rightarrow P_1$, $2 \rightarrow P_2$, $2 \rightarrow P_1$, carichi $(7, 5)$, makespan **7**; l'ottimo $\{3, 3\} \mid \{2, 2, 2\}$ dà **6**. ×
- *Ordine di input*: può ricevere in ingresso proprio i due casi precedenti, ereditandone i controesempi. ×

⚠ Nota del docente: dedurre l'ottimalità "perché funziona su un esempio" è un errore concettuale grave — un solo esempio in cui fallisce la confuta, ma nessun numero di esempi favorevoli la dimostra.

minproduct: prodotto minimo di un sottoinsieme [22-07-13 · B1]

Traccia. Minimo prodotto ottenibile moltiplicando un sottoinsieme non vuoto di A .

Soluzione greedy (corretta). Se ci sono valori negativi si moltiplicano tutti i non nulli usando un **numero dispari** di negativi (se sono pari, si esclude il negativo più vicino a zero); se sono tutti ≥ 0 si restituisce il minimo valore.

Complessità. $O(n)$.

Correttezza. I valori sono interi, quindi ogni non nullo ha $|v| \geq 1$: includere un elemento **non riduce mai** |prodotto|. Sia q il numero di negativi.

- Se $q \geq 1$: il prodotto può essere reso negativo, e il minimo (più negativo) è il negativo di **modulo massimo**. Il modulo è massimo prendendo **tutti** i non nulli; per avere segno negativo serve un numero **dispari** di negativi. Se q è dispari, il prodotto di tutti i non nulli è già negativo — ottimo. Se q è pari, occorre scartare un solo negativo per invertire il segno perdendo meno modulo possibile: quello di **valore assoluto minimo** (il negativo più vicino a 0). Gli zeri restano esclusi (darebbero $0 >$ un negativo).
- Se $q = 0$ (tutti ≥ 0): ogni prodotto è ≥ 0 ; con ≥ 2 fattori positivi il prodotto è $\geq$ di ciascuno, quindi il minimo si ottiene con il **singolo** elemento più piccolo (che è 0 se uno zero è presente). Restituire $\min A$ è corretto. ■

⚠ **La DP ingenua è sbagliata:** $DP[i] = \min\{DP[i-1], DP[i-1] \cdot A[i], A[i]\}$ fallisce su $[-2, -2, -2]$ (dà -2 invece di -8): non vale la sottostruttura ottima perché moltiplicare un negativo per un negativo dà un positivo "utile dopo". La DP corretta tiene **sia max che min** correnti.

F. Backtracking ed enumerazione

Seconda categoria per frequenza (33 esercizi, **33%**): in pratica **ogni esame** ha un esercizio "stampa tutte le X". Schema base: `rec(S, i, ...)` con (1) **condizione di terminazione** → stampa; (2) ciclo sulle **scelte ammissibili** per la posizione i ; (3) registrazione della scelta e ricorsione. La complessità è quasi sempre **superpolinomiale** e va dichiarata: spesso si può fare **pruning** (lo si sottolinea, e il docente lo premia). Attenzione a stampare ogni soluzione **una volta sola**.

Scheletro tipico vs diverso. I **template canonici** da riusare sono tre: `printBits` (riempi la stringa posizione-per-posizione, F.1), `printIncreasing` (prendi/non-prendi con stack, F.2), `allPaths` (DFS radice-foglia con stack del cammino, F.6); F.4 e F.5 descrivono il proprio template a parole nell'intro (permutazioni per **swap**; DFS con `visited` da **smarcare** al backtrack). Riconosciuto il template, ogni esercizio cambia solo la **condizione di ammissibilità** e lo **stato** passato nella ricorsione.

Pseudocodice. Per rendere il file autosufficiente, sotto ogni esercizio è riportato il codice ricostruito dalla **soluzione ufficiale** dell'esame corrispondente (stile del corso, indici 1-based, `stampa` = output). ⚠ Nei problemi su grafo/griglia il punto critico è sempre lo stesso: **smarcare** la cella/il nodo al ritorno (`visited` = `false` / ripristino del valore) — la sua assenza è l'errore più penalizzato dal docente.

F.1 Stringhe binarie / n-arie con vincoli

Tutte varianti dello stesso backtracking: si riempie la stringa posizione per posizione, con una condizione che filtra la scelta del prossimo carattere in base al precedente / a un contatore. Il numero di stringhe prodotte segue spesso Fibonacci o simili.

printBits: niente 1 consecutivi [21-07-26 · B2]

Traccia. Stampare tutte le stringhe di n bit senza due 1 consecutivi. Si mette sempre 0; si mette 1 solo se il precedente non è 1. **Complessità.** $O(n \cdot f(n))$ (f = Fibonacci; limite lasco $O(n2^n)$).

```
printBits(S, i, n, prev):
    if i > n:
        stampa(S)
        return
    for c in {0, 1}:
        if c == 0 or prev != 1:      # ← AMMISSIBILITÀ
            S[i] = c
            printBits(S, i+1, n, c)  # ← passo lo STATO aggiornato
```

printBitsK: al più k uni consecutivi [21-08-31 · B1]

Traccia. Stringhe binarie di n bit con al più k uni consecutivi. Stato aggiuntivo c_1 = uni ancora inseribili; si azzerà a k quando si mette uno 0. **Complessità.** $O(n2^n)$ nel caso $k = n$.

```
printBits(n, k):
    S[1..n]
    printRec(S, n, k, k)           # i parte da n; c1 = k uni consecutivi
    disponibili

printRec(S, i, c1, k):
    if i == 0:
        stampa(S)
    else:
        S[i] = 0                   # 0 sempre lecito → ripristina c1 a k
        printRec(S, i-1, k, k)
        if c1 > 0:                  # 1 solo se restano uni consecutivi
            S[i] = 1
            printRec(S, i-1, c1-1, k)
```

binary: $\leq n_0$ zeri e $\leq n_1$ uni consecutivi [22-02-07 · B1]

Traccia. Come sopra ma con due tetti (0 e 1). Due contatori i_0, i_1 che si re-inizializzano scegliendo il bit opposto. **Complessità.** $O(n2^n)$.

```
binary(n, n0, n1):
    S[1..n]
    bRec(S, n, n0, n1, n0, n1)    # i0, i1 = 0/1 consecutivi ancora inseribili

bRec(S, i, i0, i1, n0, n1):
    if i == 0:
        stampa(S)
    if i0 > 0:                      # metto 0 → ripristina i1 a n1
        S[i] = 0
```

```

    bRec(S, i-1, i0-1, n1, n0, n1)
  if i1 > 0:                                # metto 1 → ripristina i0 a n0
    S[i] = 1
    bRec(S, i-1, n0, i1-1, n0, n1)

```

printOctals: cifre ottali non uguali consecutive [22-01-17 · B1]

Traccia. Numeri ottali di n cifre senza due cifre uguali consecutive. Si passa *prev* e si scelgono le cifre $\neq$ *prev*.

Complessità. $\Theta(n \cdot 7^n)$ (prima cifra 8 scelte, poi 7).

```

octals(n):
  S[1..n]
  octalsRec(S, n, -1)                    # prev = -1: la prima cifra può essere 0..7

octalsRec(S, i, prev):
  if i == 0:
    stampa(S)
  else:
    for d = 0 to 7:
      if d != prev:                      # ← AMMISSIBILITÀ: cifra ≠ dalla precedente
        S[i] = d
        octalsRec(S, i-1, d)

```

printNumbers: numeri con cifre tutte distinte [25-02-03 · B1]

Traccia. Stampare i numeri in $[0, 10^n)$ senza cifre ripetute.

Idea. Backtracking che costruisce il numero cifra per cifra: un array booleano *set* ricorda quali delle cifre $0..9$ sono ancora libere, e a ogni posizione si prova ogni cifra ancora disponibile (marca → ricorri → smarca). Due accortezze: (1) per evitare lo **zero iniziale** la prima cifra si sceglie fra $1..9$ (lo 0 singolo si stampa a parte); (2) si stampa il numero parziale **a ogni chiamata**, così escono anche i numeri con meno di n cifre.

Perché la complessità è costante. Un numero **senza cifre ripetute** usa cifre distinte prese da $\{0..9\}$: non può quindi avere **più di 10 cifre**. Per quanto grande sia n , il backtracking non scende mai oltre profondità 10 (all'11ª cifra *set* è tutto occupato e il ramo muore), e il totale dei numeri stampati è limitato dalla quantità — **fissa** — di interi a cifre distinte: sommando su lunghezze $1..10$ si ottengono $10 + 81 + 648 + \dots \approx 8,9$ milioni di numeri, **indipendente da n** . Poiché non dipende da n , il costo è $\Theta(1)$ rispetto a n . (Stime più lasche ma accettate: $O(10^n)$ guardando solo la ricorsione, o $O(n 10^n)$ contando la stampa — entrambe ignorano il tetto delle 10 cifre.)

```

printNumbers(n):
  set[0..9] = {true, ..., true}          # tutte le cifre libere
  S[1..min(10,n)]                        # ← al più 10 posizioni, mai di più
  print 0                                 # lo zero singolo, a parte
  for k = 1 to 9:                          # prima cifra ≠ 0
    S[1] = k; set[k] = false
    pnRec(n, S, 2, set)
    set[k] = true                          # backtrack

pnRec(n, S, i, set):
  print S[1..i-1]                         # stampa anche i numeri più corti
  if i <= n:
    for k = 0 to 9:
      if set[k]:                          # ← AMMISSIBILITÀ: cifra ancora libera
        S[i] = k; set[k] = false

```

```
pnRec(n, S, i+1, set)
set[k] = true          # backtrack
```

numeri zerabili: più 0 che 1 (con pruning) [26-01-12 · B1]

Traccia. Stampare le stringhe di n bit con **strettamente più zeri che uni**. Si mette sempre 0; si mette 1 solo se resta possibile avere più 0 che 1, cioè se $2(\text{ones} + 1) < n$. **Complessità.** $\Theta(2^{n-1})$ grazie al pruning (senza pruning: solo 90% del voto).

printPar: parentesizzazioni corrette [21-01-18 · B1]

Traccia. Stampare tutte le stringhe ben bilanciate di n coppie. Si tengono **toOpen**, **toClose**, **open**: si apre se **toOpen**>0, si chiude se **toClose**>0 $\wedge$ **open**>0. **Complessità.** $\Omega(\frac{4^n}{\sqrt{n}})$ (numero di **Catalan**). (Il conteggio è in B.2, [25-01-14 · B3].)

```
printPar(n):
    S[1..2n]
    printParRec(S, 0, n, n, 1)          # open, toOpen, toClose, i

printParRec(S, open, toOpen, toClose, i):
    if toOpen == 0 and toClose == 0:
        stampa(S)
    else:
        if toOpen > 0:                  # posso aprire
            S[i] = "("
            printParRec(S, open+1, toOpen-1, toClose, i+1)
        if toClose > 0 and open > 0:    # posso chiudere
            S[i] = ")"
            printParRec(S, open-1, toOpen, toClose-1, i+1)
```

printWithSpaces: inserire spazi non consecutivi [22-07-13 · B2]

Traccia. Stampare tutti i modi di inserire zero o più spazi in una stringa, senza due spazi consecutivi. Ogni carattere (tranne il primo) è preceduto o no da uno spazio $\rightarrow$ maschera booleana di $n - 1$ bit. **Complessità.** $O(n \cdot 2^{n-1})$. (Il vincolo "niente spazi consecutivi" è gratis: lo spazio sta solo nel «buco» prima di un carattere, e i buchi sono separati da caratteri.)

```
printWithSpaces(A, n):
    S[1..n]                                # S[i] = true  $\Leftrightarrow$  metti uno spazio prima di A[i]
    swRec(A, S, n)

swRec(A, S, i):
    if i == 1:                             # maschera completa (S[2..n] decisi)
        stampaConSpazi(A, S, n)
    else:
        S[i] = true                         # spazio prima di A[i]
        swRec(A, S, i-1)
        S[i] = false                        # nessuno spazio
        swRec(A, S, i-1)

stampaConSpazi(A, S, n):
    stampa(A[1])                            # il primo carattere mai preceduto da spazio
    for j = 2 to n:
```

```
if S[j]: stampa(" ")
stampa(A[j])
```

F.2 Sottosequenze e sottoinsiemi

Backtracking "prendi / non prendi" su ogni elemento, con un vincolo sull'ultimo preso (crescente, distanza, alternanza). Tipicamente con uno **stack** riempito dal fondo. Nel caso pessimo (input ordinato) si stampano tutti i 2^n sottoinsiemi $\rightarrow O(n2^n)$.

printIncreasing: tutte le sottosequenze crescenti [20-09-04 · B3]

Traccia. Stampare tutte le sottosequenze crescenti di A . Stack riempito dal fondo: si spinge $A[i]$ solo se $<$ della cima. **Complessità.** $O(n \cdot 2^n)$.

```
printIncreasing(A, n, i, S, k):
    if i > n:
        stampa(S[1..k])
        return
    printIncreasing(A, n, i+1, S, k)          # non prendo: sempre lecito
    if k == 0 or A[i] > S[k]:                 # ← AMMISSIBILITÀ
        S[k+1] = A[i]
        printIncreasing(A, n, i+1, S, k+1)    # prendo
```

printKSequence: tutte le sottosequenze k -limitate [21-08-31 · B2]

Traccia. Stampare le sottosequenze in cui valori consecutivi differiscono $\leq k$. Come sopra ma con condizione $|A[i] - \text{cima}| \leq k$. **Complessità.** $O(n \cdot 2^n)$.

```
printKSequence(A, n, k):
    S = Stack()
    ksRec(A, k, n, S)

ksRec(A, k, i, S):
    if i == 0:
        stampa(S)                                # ordine LIFO
    else:
        if S.isEmpty() or |A[i] - S.top()| <= k: # ← AMMISSIBILITÀ
            S.push(A[i])                          # prendo A[i]
            ksRec(A, k, i-1, S)
            S.pop()
        ksRec(A, k, i-1, S)                       # non prendo A[i]
```

zigzag: tutte le sottosequenze a zig-zag [24-08-26 · B3]

Traccia. Stampare le sottosequenze alternanti ($<$, $>$, $<$, ... sugli indici della sottosequenza). Si passa **dir** e si accetta $A[i]$ se $(A[i] - \text{ultimo}) \cdot \text{dir} < 0$, invertendo **dir**. **Complessità.** $O(2^n)$.

```
printZigZag(A, n):
    S[1..n]
    zzRec(A, n, S, 1, 1, 1)                      # iA = indice in A, iS = indice in S, dir = +1

zzRec(A, n, S, iA, iS, dir):
```

```

if iA > n:
    stampa(S[1..iS-1])
else:
    zzRec(A, n, S, iA+1, iS, dir)           # non prendo A[iA]
    if iS == 1 or (A[iA] - S[iS-1]) * dir < 0: # primo, o segno giusto
        S[iS] = A[iA]
        zzRec(A, n, S, iA+1, iS+1, -dir)    # prendo e inverto dir

```

tuple ben ordinate: crescenti o decrescenti fino a lunghezza k [25-06-13 · B3]

Traccia. Stampare tutte le tuple (di lunghezza $2..k$) strettamente crescenti o decrescenti. Due ricorsioni gemelle (up e down) con pruning sull'ordine. **Complessità.** $O(n \cdot 2^n)$.

```

printTuples(A, n, k):
    S[1..k]
    printRecUp(A, n, k, S, 1, 1)          # tuple crescenti
    printRecDown(A, n, k, S, 1, 1)        # tuple decrescenti (identica con S[i-1] > A[j])

printRecUp(A, n, k, S, i, j):             # i = pos. in S da riempire, j = indice in A
    if i <= k and j <= n:
        printRecUp(A, n, k, S, i, j+1)    # salto A[j]
        if i == 1 or S[i-1] < A[j]:         # crescente
            S[i] = A[j]
            if i > 1: stampa(S[1..i])        # stampa (solo tuple di
            lunghezza >= 2)
        printRecUp(A, n, k, S, i+1, j+1)

```

sottoinsiemi k -distanti [25-08-25 · B2]

Traccia. Stampare i sottoinsiemi in cui ogni coppia dista $> k$. Si ordina e si rimuovono i duplicati, poi si costruisce in ordine crescente spingendo $A[i]$ solo se cima - $A[i] > k$. **Complessità.** $O(n \cdot 2^n)$.

```

printDistantSets(A, n, k):
    sort(A, n)
    m = 0                                     # rimuovo i duplicati → m valori distinti
    for i = 1 to n:
        if m == 0 or A[i-1] != A[i]:
            m = m+1; A[m] = A[i]
    dsRec(A, m, k, Stack())

dsRec(A, i, k, S):
    if i == 0:
        stampa(S)
    else:
        if S.isEmpty() or S.top() - A[i] > k: # costruzione decrescente: gap > k
            S.push(A[i])                       # prendo A[i]
            dsRec(A, i-1, k, S)
            S.pop()
        dsRec(A, i-1, k, S)                   # non prendo A[i]

```

minCardinality: sottoinsiemi con minimo = cardinalità [23-01-17 · B3]

Traccia. Stampare i sottoinsiemi non vuoti di $\{1, \dots, n\}$ il cui **minimo è uguale alla dimensione**. Il primo elemento k fissa cardinalità e minimo; si scelgono altri $k - 1$ valori $> k$. Vettore di dimensione $\lceil n/2 \rceil$ (serve $k - 1 \leq n - k$).

Complessità. superpolinomiale (il numero di insiemi è Fibonacci).

```
minCardinality(n):
  S[1..ceil(n/2)]
  for v = 1 to n:
    S[1] = v
    mcRec(S, n, 1)

mcRec(S, n, i):
  if i == S[1]:
    stampa(S[1..i])
  else:
    for j = S[i]+1 to n:
      S[i+1] = j
      mcRec(S, n, i+1)
```

S[1] = v fissa minimo = cardinalità = v
1 elemento già inserito
i = numero di elementi già inseriti
raggiunta la cardinalità richiesta (= minimo
prossimo valore > ultimo inserito

lessThanHalf: sequenze che dimezzano [24-02-01 · B1]

Traccia. Stampare le sequenze che iniziano con n e in cui ogni elemento è $\leq$ metà del precedente. Si stampa a ogni chiamata (anche le corte), poi si ricorre con ogni $i \in [1, \lfloor \text{top}/2 \rfloor]$. **Complessità.** superpolinomiale ($T(n) = 1 + \sum_{i=1}^{\lfloor n/2 \rfloor} T(i)$, dimostrabile $\Omega(n^k) \forall k$).

```
lessThanHalf(n):
  stack = Stack(); stack.push(n)
  lthRec(stack)

lthRec(stack):
  stampa(stack)
  for i = 1 to floor(stack.top()/2):
    stack.push(i)
    lthRec(stack)
    stack.pop()
```

n è sempre il primo elemento
← stampa a OGNI chiamata (anche le sequenze corte)
prossimo valore $\leq$ metà dell'ultimo inserito
⚠ backtrack

F.3 Composizioni e somme di interi

generate: tutte le somme palindrome di n [20-06-12 · B3]

Traccia. Stampare tutte le somme palindrome di n (incluse le permutazioni). Si memorizza **metà** sequenza in S , sottraendo $2j$ a ogni scelta; l'altra metà si stampa a specchio, con l'eventuale elemento centrale se **missing** > 0 .

Complessità. $O(n \cdot 2^{n/2})$.

```
generate(n):
  S[1..floor(n/2)]
  palindroma
  genRec(S, 1, n)
  centro

genRec(S, i, missing):
```

in S memorizzo solo METÀ della somma
missing = quanto resta da coprire al centro

```

for k = 1 to i-1:      stampa(S[k])      # prima metà
if missing > 0:        stampa(missing)   # eventuale addendo centrale
for k = i-1 downto 1: stampa(S[k])       # seconda metà, a specchio
for j = 1 to floor(missing/2):           # la prossima coppia esterna vale j...
    S[i] = j
    genRec(S, i+1, missing - 2*j)        # ...e consuma 2j (un j per lato)

```

primes: n come somma di esattamente k primi distinti [21-02-08 · B1]

Traccia. Dato il vettore ordinato dei primi $\leq n$, stampare tutti i modi di scrivere n come somma di k primi distinti. Backtracking "prendi/non prendi" $P[p]$, decrementando k e n . **Complessità.** $O\left(\binom{p}{k}\right)$.

```

primes(P, p, k, n):           # P = primi ordinati; p = quanti primi restano
    S[1..k]                   # k = termini richiesti, n = somma da comporre
    primesRec(P, S, p, k, n)

primesRec(P, S, p, k, n):
    if n == 0 and k == 0:      # somma raggiunta con esattamente k termini
        stampa(S)
    else if p > 0 and k > 0 and n > 0:
        primesRec(P, S, p-1, k, n)      # non uso P[p]
        S[k] = P[p]                     # uso P[p]
        primesRec(P, S, p-1, k-1, n-P[p])

```

mathWays: stampa tutti i modi $k \rightarrow n$ [22-06-21 · B1]

Traccia. Stampare tutte le sequenze che portano da k a n con $+1, \times 2, \cdot^2$. Si costruisce S applicando le tre operazioni; si taglia se si supera n . **Complessità.** $\approx O(n \cdot 3^n)$ (molto sovrastimata: le potenze fanno crescere in fretta). (Il conteggio è in B.1, [22-02-07 · B3].)

```

mathWays(k, n):
    S[1..n]
    mwRec(S, 1, k, n)

mwRec(S, i, k, n):
    S[i] = k
    if k == n:                               # raggiunto n → stampa la sequenza
        stampa(S[1..i])
    else if k < n:                             # ← PRUNING: se k > n lo stato è
        irrecuperabile                         # ← PRUNING: se k > n lo stato è
        mwRec(S, i+1, k+1, n)                 # +1
        mwRec(S, i+1, 2*k, n)                 # ×2
        mwRec(S, i+1, k*k, n)                 # ·²

```

F.4 Permutazioni con vincoli

Si adatta l'algoritmo di generazione delle permutazioni visto a lezione, filtrando le scelte con il vincolo del problema. Costo tipico $O(n \cdot n!)$.

printElegant: permutazioni "elegant" [22-08-24 · B2]

Traccia. Permutazioni di $1..n$ in cui ogni a_i è divisibile per i o viceversa. Si permuta scegliendo per la posizione i solo i valori c con $c \bmod i = 0 \vee i \bmod c = 0$. **Complessità.** $O(n \cdot n!)$ (con la versione a swap; $O(n^2 n!)$ con copie

dell'insieme).

```
printElegant(n):
    S[1..n]; for i = 1 to n: S[i] = i
    peRec(S, n)

peRec(S, i):
    # fissa le posizioni da i giù a 2, permutando
    per swap
    if i == 1:
        stampa(S)
        # la posizione 1 è sempre elegante (tutto
    divisibile per 1)
    else:
        for j = 1 to i:
            swap(S, i, j)
            # porto S[j] in posizione i
            if S[i] mod i == 0 or i mod S[i] == 0:
                # elegante in posizione i?
                peRec(S, i-1)
            swap(S, i, j)
            # ⚠️ ripristina
    (backtrack)
```

magic square: tutti i quadrati magici $n \times n$ [24-06-14 · B3]

Traccia. Stampare tutti i quadrati magici di lato n . Si generano le permutazioni di $1..n^2$ nella matrice e si verifica con **isMagic**. **Pruning:** appena una riga è completa, se la sua somma $\neq n(n^2 + 1)/2$ si pota. **Complessità.** $O(n^2(n^2)!)$.

```
printMagic(n):
    A[1..n][1..n]
    left = Set({1..n^2})
    # valori ancora da piazzare
    pmRec(A, n, 1, 1, left)

pmRec(A, n, i, j, left):
    if j > n:
        # riga i completata
        if sumRiga(A, i) != n*(n^2+1)/2:
            # ← PRUNING: somma di riga già
            sbagliata
        return
    j = 1; i = i+1
    # passo alla riga successiva
    if i > n:
        # matrice piena = una permutazione
        if isMagic(A, n): stampa(A)
        return
    for k in left:
        # (iterando su una copia di left)
        A[i][j] = k; left.remove(k)
        pmRec(A, n, i, j+1, left)
        left.insert(k)
        # ⚠️ ripristina (backtrack)
```

F.5 Cammini e cicli su griglie e grafi

Backtracking su un grafo/griglia: si marca il nodo visitato, si ricorre sui vicini, si smarca al ritorno. Serve **sempre** ricordarsi di (i) smarcare al backtrack e (ii) controllare la condizione di chiusura giusta. Il costo pessimo (grafo completo) è $O(n!)$.

longestPath: cammino di 1 più lungo da $(1, 1)$ [20-01-14 · B3]

Traccia. Massima lunghezza di un cammino di celle-1 adiacenti da $(1, 1)$ (è il *longest path*, NP-hard). Si marca la cella a -1 durante la visita, si esplorano le 4 direzioni, si ripristina. **Complessità.** $O(4^{nm})$.

```

longestPath(M, n, m):
    return lpRec(M, n, m, 1, 1)

lpRec(M, n, m, i, j):
    if 1 <= i <= n and 1 <= j <= m and M[i][j] == 1:
        M[i][j] = -1                # marca "visitata"
        best = 1 + max(lpRec(M,n,m,i+1,j), lpRec(M,n,m,i-1,j),
                        lpRec(M,n,m,i,j+1), lpRec(M,n,m,i,j-1))
        M[i][j] = 1                # ⚠️ ripristina (backtrack)
        return best
    else:
        return 0                    # fuori griglia o cella non percorribile

```

countPaths: contare i cammini con salti = valore cella [20-02-04 · B1]

Traccia. Contare i cammini semplici da $(1, 1)$ a (n, n) dove da (i, j) si salta di $A[i][j]$ celle. Backtracking con matrice **visited** (i sottoproblemi si muovono avanti e indietro → **niente DP**). **Complessità.** $O(4^{n^2})$ (molto potata in pratica).

```

countPaths(A, n):
    visited[1..n][1..n] = false
    return cpRec(A, visited, n, 1, 1)

cpRec(A, visited, n, i, j):
    if i == n and j == n:
        return 1                    # un cammino semplice
    completato
    else if 1 <= i <= n and 1 <= j <= n and not visited[i][j]:
        visited[i][j] = true
        ret = cpRec(A, visited, n, i+A[i][j], j) + cpRec(A, visited, n, i-A[i][j], j)
              + cpRec(A, visited, n, i, j+A[i][j]) + cpRec(A, visited, n, i, j-A[i][j])
        visited[i][j] = false        # ⚠️ ripristina (backtrack)
        return ret
    else:
        return 0                    # fuori griglia o già visitata

```

all paths: cammini che coprono tutte le celle libere [25-07-14 · B3]

Traccia. Stampare i cammini da 1 a 2 (4 direzioni) che visitano **ogni** cella libera esattamente una volta (Hamiltoniano su griglia). Contatore **remaining** delle celle da visitare; si arriva a 2 solo con **remaining=0**. Si marca la cella a -1 e si ripristina. **Complessità.** $O(n^2 3^{n^2})$.

```

printAllPaths(A, n):
    remaining = 0; (sr, sc) = (0, 0)
    for r = 1 to n, c = 1 to n:
        if A[r][c] == 0: remaining = remaining + 1    # celle libere da coprire
        else if A[r][c] == 1: (sr, sc) = (r, c)      # partenza
    apRec(A, n, sr, sc, remaining, Stack())

apRec(A, n, r, c, remaining, S):
    if 1 <= r <= n and 1 <= c <= n and A[r][c] != -1:
        if A[r][c] == 2:                            # destinazione
            if remaining == 0:                        # ...raggiunta coprendo tutto
                S.push((r,c)); stampa(S); S.pop()
        else:

```

```

A[r][c] = -1; S.push((r,c)) # marca e registra
apRec(A, n, r+1, c, remaining-1, S)
apRec(A, n, r-1, c, remaining-1, S)
apRec(A, n, r, c+1, remaining-1, S)
apRec(A, n, r, c-1, remaining-1, S)
A[r][c] = 0; S.pop() # ⚠️ ripristina (backtrack)

```

10.000 passi / New York Dreaming: cammini di lunghezza vincolata [23-07-11 · B2] [26-02-02 · B1]

Traccia. Stampare i cammini (chiusi in 26, o da **src** a **dest**) la cui lunghezza cade in un intervallo dato. Backtracking che **non torna indietro** sull'arco appena percorso e **pota** appena la distanza residua esce dal range (pesi positivi).

- [23-07-11 · B2]: da **src** a **dest** con esattamente 5000 passi; poiché i pesi sono ≥ 1 , i passi sono ≤ 5000 → complessità **polinomiale** $O(n^{5000})$.
- [26-02-02 · B1]: cammini chiusi da **s** a **s** (senza ripetere nodi) di lunghezza in $[k - 500, k + 500]$; complessità $O(n \cdot n!)$.

[23-07-11 · B2] — trova **un** cammino (ritorna **boolean**), senza vettore **visited**: basta non tornare sull'arco appena percorso. **ln** = passi ancora da fare.

```

searchPath(G, src, dest):
    S = Stack(); S.push(src)
    return spRec(G, src, nil, dest, S, 5000)

spRec(G, curr, prev, dest, S, ln):
    if curr == dest and ln == 0:
        stampa(S); return true
    else if ln > 0:
        for next in curr.adj() - {prev}: # non torno sull'arco appena
            fatto
                S.push(next)
                if spRec(G, next, curr, dest, S, ln - w(curr,next)): return true
                S.pop() # ⚠️ backtrack
    return false

```

[26-02-02 · B1] — **enumera tutti** i cicli chiusi in **s**, con **visited** per la semplicità e **missingDist** = distanza mancante rispetto a **k**; pruning se non si può più chiudere nel range.

```

printPaths(G, s, k):
    visited = Set({s}); path = Stack(); path.push(s)
    ppRec(G, s, s, k, visited, path) # missingDist parte da k

ppRec(G, u, s, missingDist, visited, path):
    for v in G.adj(u):
        if v == s: # tento di richiudere il ciclo
            if path.size() > 1 and -500 <= missingDist - w(u,s) <= 500:
                path.push(s); stampa(path); path.pop()
            else if v not in visited and missingDist - w(u,v) >= -500: # ← PRUNING
                visited.insert(v); path.push(v)
                ppRec(G, v, s, missingDist - w(u,v), visited, path)
                path.pop(); visited.remove(v) # ⚠️ backtrack

```

Cicli / cammini Hamiltoniani [20-07-24 · B3] [24-07-02 · B2] [24-01-09 · B2]

Traccia. Enumerare/decidere strutture hamiltoniane con backtracking-DFS:

- [20-07-24 · B3] — **stampare** tutti i cicli hamiltoniani di un grafo orientato, partendo da un nodo fisso (per non stamparne n copie), controllando l'arco di ritorno. $O(n!)$.
- [24-07-02 · B2] — **decidere** se esiste un ciclo hamiltoniano (**boolean**); si estende il cammino sui vicini non visitati, chiudendo quando **len** = **n** e c'è l'arco verso il nodo 1. $O(n!)$.
- [24-01-09 · B2] — **stampare tutti i cicli semplici** che passano per un nodo **start** (cicli orientati distinti); si stampa lo stack quando si reincontra **start**. $\Theta(n!)$ nel caso pessimo.

[20-07-24 · B3] — **stampa** tutti i cicli hamiltoniani, partendo da un nodo **fisso** (così ogni ciclo esce una volta sola) e controllando l'arco di ritorno.

```
printHamilton(G):
    visited[1..G.n] = false
    path[1..G.n]
    hamRec(G, 1, 1, path, visited)           # nodo di partenza FISSO = 1

hamRec(G, u, i, path, visited):
    path[i] = u
    if i == G.n:                             # visitati tutti i nodi...
        if path[1] in G.adj(u): stampa(path) # ...e c'è l'arco di ritorno
    else:
        visited[u] = true
        for v in G.adj(u):
            if not visited[v]:
                hamRec(G, v, i+1, path, visited)
        visited[u] = false                    # ⚠ backtrack
```

[24-07-02 · B2] — **decide** se esiste un ciclo hamiltoniano (**boolean**), chiudendo appena **len** == **n** con l'arco di ritorno.

```
hamilton(G):
    visited[1..G.n] = false
    return visitRec(G, 1, 1, visited)         # len = 1, dal nodo 1

visitRec(G, len, u, visited):
    if len == G.n and 1 in G.adj(u):         # tutti visitati + arco di ritorno
        return true
    else:
        visited[u] = true
        for v in G.adj(u):
            if not visited[v]:
                if visitRec(G, len+1, v, visited): return true
        visited[u] = false                   # ⚠ backtrack
    return false
```

[24-01-09 · B2] — **stampa tutti i cicli semplici** per **start**: si stampa lo stack ogni volta che un vicino è **start** (non solo a lunghezza n).

```
printCycles(G, start):
    visited[1..G.n] = false
```

```
pcRec(G, start, start, Stack(), visited)

pcRec(G, start, curr, S, visited):
    S.push(curr); visited[curr] = true
    for next in G.adj(curr):
        if next == start:                    # ciclo chiuso → stampa
            stampa(S, start)
        else if not visited[next]:
            pcRec(G, start, next, S, visited)
    visited[curr] = false; S.pop()          # ⚠ backtrack
```

⚠ Errori tipici (segnalati dal docente): dimenticare `visited[u]=false` al backtrack; dimenticare il test `len == n`; dimenticare l'arco di ritorno; usare un `Set` al posto del vettore-cammino (si perde l'ordine).

Misciotto 1: stampare tutti i mescolamenti di due stringhe [23-06-20 · B1]

Traccia. Stampare tutti gli intercalamenti (shuffle) di S_1 e S_2 che ne preservano l'ordine. Backtracking dal fondo: si aggiunge un carattere di S_1 (se resta) e/o di S_2 . **Complessità.** $O\left((n_1 + n_2) \cdot \frac{(n_1+n_2)!}{n_1!n_2!}\right)$. (La versione "verifica" è in B.5, [23-06-20 · B2].)

```
printMixing(S1, S2, n1, n2):
    T[1..n1+n2]
    pmRec(S1, S2, T, n1, n2)                # lavora dal fondo (meno parametri)

pmRec(S1, S2, T, n1, n2):
    if n1 == 0 and n2 == 0:
        stampa(T)
    else:
        if n1 > 0:                          # prossimo carattere preso da S1
            T[n1+n2] = S1[n1]
            pmRec(S1, S2, T, n1-1, n2)
        if n2 > 0:                          # prossimo carattere preso da S2
            T[n1+n2] = S2[n2]
            pmRec(S1, S2, T, n1, n2-1)
```

subPalindrome: tutte le sottosequenze palindrome [23-08-23 · B2]

Traccia. Stampare tutte le sottosequenze palindrome di S . Si generano tutte le 2^n sottosequenze e si stampa solo quelle palindrome (verifica $O(n)$). **Complessità.** $\Theta(n \cdot 2^n)$.

```
subPalindrome(str, n):
    sub[1..n]
    palRec(str, sub, n, 0)                  # i scorre str da n a 0; j = lunghezza di sub

palRec(str, sub, i, j):
    if i == 0:                             # sottosequenza completata
        if isPalindrome(sub, 1, j): stampa(sub[1..j])
    else:
        palRec(str, sub, i-1, j)            # non prendo str[i]
        sub[j+1] = str[i]                  # prendo str[i] (sub è memorizzata al
        contrario, irrilevante)
        palRec(str, sub, i-1, j+1)
```

F.6 Cammini radice-foglia su alberi

Visita in profondità con uno **stack** che accumula il cammino corrente; alla foglia si stampa, poi si fa **pop** (backtrack). È di fatto una DFS: ogni nodo visitato una volta, ma la stampa può portare il costo a $O(n^2)$.

printPositive / allpath: cammini radice-foglia [20-07-03 · B2] [25-01-14 · B2]

Traccia. Stampare tutti i cammini dalla radice alle foglie:

- [20-07-03 · B2] — solo i cammini composti da **valori positivi** (ci si ferma sul primo valore ≤ 0):

```
printAllPaths(T):
    ppRec(T, Stack())

ppRec(T, path):
    if T != nil and T.value > 0:                # ci si ferma sul primo valore ≤ 0
        path.push(T.value)
        if T.left == nil and T.right == nil:
            stampa(path)                       # foglia → stampa il cammino
        else:
            ppRec(T.left, path)
            ppRec(T.right, path)
        path.pop()                             # ⚠ backtrack
```

- [25-01-14 · B2] — **tutti** i cammini radice-foglia. Con soluzione:

```
allPaths(r, c, visited, S, remaining):
    visited[r][c] = true                        # marco
    S.push((r,c))
    if remaining == 1:
        stampa(S)
    else:
        for (r2,c2) in vicini(r,c):
            if not visited[r2][c2]:
                allPaths(r2, c2, visited, S, remaining-1)
    S.pop()
    visited[r][c] = false                       # ⚠ SMARCO
```

Complessità. $O(nh)$; nel caso pessimo (catena lineare + sottoalbero completo in fondo) si raggiunge $O(n^2)$; su albero bilanciato $O(n \log n)$.

G. Array, scansioni, ricerca

Esercizi che **non richiedono una tecnica avanzata**: una scansione lineare o una riduzione a un algoritmo di lezione. Sono in genere i **B1** (i più leggeri). Il punto è **riconoscere** che non serve nient'altro e giustificare correttezza e complessità.

Scheletro tipico vs diverso. Tutti gli esercizi hanno scheletro tipico: una **scansione lineare** o una **riduzione a un algoritmo di lezione** (Kadane, finestra a due indici). Nessun codice dedicato — il valore sta nel riconoscere che non serve altro.

zeroUnbalance: massimo sbilanciamento 0/1 [20-02-04 · B2]

Traccia. Massimo di (#zeri – #uni) fra i sottovettori contigui. Si trasforma $0 \rightarrow +1$, $1 \rightarrow -1$ e si applica **somma massimale** (Kadane). **Complessità.** $\Theta(n)$. (Anche in [B.7](#).)

connect-x: più lunga linea contigua di **true** [\[24-08-26 · B1\]](#)

Traccia. Più lunga sequenza contigua orizzontale o verticale di valori **true** in una matrice. Doppia scansione (righe, poi colonne) con un contatore. **Complessità.** $\Theta(n^2)$.

countArithmetic: sottovettori aritmetici massimali [\[25-01-14 · B1\]](#)

Traccia. Contare i sottovettori **contigui** aritmetici massimali (≥ 3 elementi, differenza costante). Scansione unica confrontando la differenza corrente con la precedente; si conta un nuovo sottovettore quando la stessa differenza compare per la 2^a volta. **Complessità.** $O(n)$.

minRemove: minime rimozioni per $\max - \min \leq k$ [\[21-01-18 · B2\]](#)

Ordina + finestra scorrevole a due indici, $O(n \log n)$. Descritto in [E.1](#).

Appendice A: indice per esame

Per ritrovare un esercizio partendo dalla data del compito. ([\[X\]](#) = categoria; $\rightarrow$ = cross-riferimento a una categoria "duale".)

Esame	B1	B2	B3
20-01-14	Catena di coppie = interval scheduling [E.1]	Sottovettore pari max, Kadane [B.7]	Longest path di 1 in matrice [F.5]
20-02-04	Contare cammini con salti [F.5]	Zero-sbilanciamento = maxsum [G]	Quadrato con bordi di 1 [B.3]
20-06-12	maxGain $2 \times n$ = Hateville [B.7]	Cammini griglia $n \times n$ [B.3]	Stampa somme palindrome [F.3]
20-07-03	Permutazione palindroma max (greedy) [E.1]	Cammini radice-foglia positivi [F.6]	Sottoseq. k -limitata max [B.7]
20-07-24	LCS di tre stringhe [B.5]	Flusso: didattica pandemia [D.2]	Cicli hamiltoniani (stampa) [F.5]
20-09-04	countMenu subset-sum conteggio [B.4]	Longest common substring [B.5]	Stampa sottoseq. crescenti [F.2]
21-01-18	Stampa parentesizzazioni [F.1]	minRemove ($\max - \min \leq k$) [G] / [E.1]	Conta alberi = Catalan [B.2]
21-02-08	Stampa somme di k primi [F.3]	Zaino leggero (min subset) [B.6]	Permutazione $</>$ (greedy) [E.1]
21-06-16	Stampa cammini griglia [F.5]	removeSum (duale zaino) [B.6]	Sottoseq. crescente somma max [B.7]
21-07-26	Punto d'incontro (cammini minimi) [C.1]	Stampa bit senza 1 consec. [F.1]	Sottoseq. k -contigua max [B.7]
21-08-31	Stampa bit $\leq k$ uni consec. [F.1]	Stampa sottoseq. k -limitate [F.2]	Conta somme palindrome pari [B.1]

Esame	B1	B2	B3
22-01-17	Stampa ottali no cifre uguali [F.1]	Longest repeating subseq. [B.5]	Conta alberi ben-bilanciati [B.2]
22-02-07	Stampa bit $\leq n_0/\leq n_1$ consec. [F.1]	Discordville (house-robber) [B.7]	mathWays (conteggio) [B.1]
22-06-21	Stampa modi mathWays [F.3]	Flusso: piano strategico [D.2]	Double sequence (conteggio) [B.1]
22-07-13	Prodotto minimo (greedy + DP) [E.2]	Stampa con spazi [F.1]	Conta sottoseq. crescenti- k [B.4]
22-08-24	Flusso: programmazione esami [D.2]	Permutazioni eleganti [F.4]	Connessioni non intersec. = LCS [B.5]
23-01-17	Griglia lineare (Tetranacci) [B.1]	Tape: greedy da smontare [E.2]	minCardinality (stampa insiemi) [F.2]
23-02-09	MCS (LCS pesata) [B.5]	Flusso: Facoltiadi (genere) [D.2]	k -prodotto (conteggio) [B.4]
23-06-20	Misciotto 1 (stampa shuffle) [F.5]	Misciotto 2 (verifica shuffle) [B.5]	Flusso: Conferenza (reviewer) [D.2]
23-07-11	Flusso: SWERC (diversità) [D.2]	10.000 passi (cammino vincolato) [F.5]	Made-up (conta alberi) [B.2]
23-08-23	Flusso: rete idrica (test archi) [D.4]	Stampa sottoseq. palindrome [F.5]	Stringhe eleganti (min inversioni) [B.7]
24-01-09	Flusso: commissioni (≥ 2 esperti) [D.2]	Stampa cicli per start [F.5]	Conta sottomatrici quadrate [B.3]
24-02-01	lessThanHalf (stampa) [F.2]	Conta alberibelli (full binary) [B.2]	LCNCS (LCS non consecutiva) [B.5]
24-06-14	Corsa a ostacoli (conta cammini) [B.3]	Multi core: greedy da smontare [E.2]	Magic square (stampa) [F.4]
24-07-02	Flusso: vie di fuga (node split) [D.3]	Hamilton (decisione) [F.5]	Norma L_1 max (LCS-like) [B.5]
24-08-26	connect-x (linea più lunga) [G]	Formazione stringhe (min mosse) [B.7]	Stampa sottoseq. zig-zag [F.2]
25-01-14	Sottovettori aritmetici [G]	Cammini radice-foglia [F.6]	Conta parentesizzazioni = Catalan [B.2]
25-02-03	Stampa numeri cifre distinte [F.1]	Giardinaggio (greedy matching) [E.1]	minSteps griglia (memoization) [B.7]
25-06-13	Flusso: pulizie (cap. m) [D.1]	Dollar word (Hexacosanacci) [B.1]	Tuple ben ordinate (stampa) [F.2]
25-07-14	Livelli bonus (salti 1/2) [B.7]	Massima espressività (+/x) [B.8]	All paths (griglia hamiltoniana) [F.5]
25-08-25	Flusso: mensa (3 categorie) [D.1]	Sottoinsiemi k -distanti [F.2]	Going nowhere (Motzkin) [B.1]

Esame	B1	B2	B3
26-01-12	Numeri zerabili (pruning) [F.1]	Flusso: junior/senior (quota) [D.2]	Borsa: k operazioni [B.7]
26-02-02	New York Dreaming (cammini) [F.5]	Flusso: personale ospedaliero [D.2]	Hateville PNRR (colorazione) [B.7]